On the Cost of Communication in Divergent Program Evolution

Matched-compute evidence from ARC-AGI-1 that a single agent with a central library strictly dominates divergent lineages communicating over a constrained channel — and a reorientation toward skill-acquisition efficiency

Aidan Jones

23 July 2026

Source: clebran-arc-agi repository. Per-experiment READMEs at experiments/2026-07-*. The review pass for this draft is in paper/reviews/, and the consolidated change log in paper/review-summary.md.

Contents

1.1 The clebran hypothesis
1.2 What the experiment found
1.3 The reorientation (preview)
1.4 Contributions
1.5 Outline
2.1 DSL program synthesis for ARC
2.2 Library learning, compression mining, and MDL
2.3 Emergent communication between divergent agents
2.4 Learning-to-Theorize and the transfer metric
2.5 Multi-agent program synthesis
2.6 Curriculum learning and machine teaching
3.1 Programs: a linear typed IR
3.2 Deterministic interpreter
3.3 GP engine
3.4 Fitness
3.5 The channel and the four payloads
3.6 Compression miner
3.7 A note on rig-evidence
4.1 Sub-hypotheses
4.2 Arms
4.3 Matched-compute protocol
4.4 Subsets
4.5 Statistics
4.6 What the experiment does not test
4.7 Configuration
5.1 The Stage-1 first test of H1/H2/H3
5.2 The frontier reachability probe
5.3 Bandwidth dose-response (H3 on Stage-1)
5.4 Recovered-primitive channel (B6/B7)
5.5 The central-library control (B5) — the headline arm
5.6 Curriculum channel (B8/B9) — even an engaged channel does not beat B5
5.7 The teach→test probe and the amortization correction

5.7.1 The teach→test probe (without a cold baseline)
5.7.2 The amortization correction (cold baseline)
5.7.3 The order experiment — the first significant process effect
5.8 Vocabulary quality
6.1 The reorientation: not what you know, how you learned it
6.2 The divergence tax
6.3 The role of measurement
6.4 Scope: what the result is for, and what it is never for
6.5 What this means for related literatures

0. Abstract

We test whether bandwidth-limited channels between divergent agents — the canonical setting of emergent communication — force compositionality and so improve transfer. The test is on a deliberately small, fully-instrumented substrate: typed programs evolved under pure reward on a 30-task frontier subset of ARC-AGI-1, with matched candidate-evaluation budgets audited to 0.10% spread across 20 seeds. The baselines are strong enough that a null is trustworthy: a single population with the same total compute, an isolated-lineage ensemble, an unconstrained library-dump channel, and — critically — a *single agent that grows a central abstraction library* (a DreamCoder/Stitch control).

The headline. On this substrate, the central-library control (B5) sits at the top of the ranking. The single significant comparison in the entire study is **B5 − B6 = +0.85 (~2.2 standard errors)** — sharing a *discovered* vocabulary between divergent agents is significantly *worse* than letting one agent keep the vocabulary. Every other multi-agent arm — across four payload types (fragment, signal, recovered-primitive, curriculum) and three constraint regimes (CH-NONE, CH-CONSTRAINED, CH-UNCONSTRAINED) — lies between the isolated-lineage floor and the central-library ceiling. The multi-agent-with-channel hypothesis is falsified by its own strongest control.

What the data also show. H1 (lineages diverge) is supported — pairwise primitive Jaccard 0.57 ± 0.10 — but divergence in this substrate is a *liability* for vocabulary quality: the single agent's mined library is more compositional (topographic similarity 0.33 vs 0.14–0.15 for the multi-agent arms) and more cross-task-reused (0.15 vs 0.05). H3 (the constraint matters) is null across all four payload types, because adaptive operator selection self-regulates a failing channel regardless of bandwidth. H2 (transfer helps) is directionally positive but not significant at 20 seeds; the recovered-primitive channel beats isolation by +0.35 (se 0.32) and the fragment channel by +0.20 (se 0.32).

What the re-scoring shows. A controlled teach→test probe with a *cold* (no-curriculum) baseline reveals that the *probe's* apparent 2× speedup was the size of the harm to a mis-targeted control; against a cold baseline, a relevant curriculum is roughly neutral (taught − cold = −986 evals, permutation p = 0.97), and an irrelevant curriculum is *slower* than not teaching. The per-student condition L + T < U fails by ~56k evals, so no amortisation across N agents can rescue it. *Students learn taught concepts* (70–78% of synthetic teaching tasks are solved by their peers) but do not transfer them to novel tasks needing them — the L2T transfer-vs-reconstruction gap at the level of whole concepts.

A first purchase on process. Holding taught *content* fixed and varying only the *order* of acquisition, a curriculum taught foundations-first significantly outperforms the same curriculum taught foundations-last (paired McNemar p = 0.021) — a dependency-violating

order halves how much of the same content is acquired (23% vs 47%). This is the study's only significant *process* effect, against a wall of null *content* effects, and it is asymmetric: *don't teach backwards* matters more than teaching in exactly the right order, because an adaptive learner self-orders when the material is merely available.

The contribution. The paper is a falsification plus a reorientation. The falsification is that, on ARC program synthesis at matched compute, a single agent with a central library strictly dominates divergent lineages with a constrained channel. The reorientation is to the question Chollet argues is the right one — the *skill-acquisition efficiency* of a system, not its skill — with a first significant process result (order matters, at the extreme) and a small set of process-level capabilities (self-directed practice, active choice of what to practise, retention without rigidity, reflection on how something was learned) that must be present before "how you learned it" can be studied richly. The asset of the work is the matched-compute measurement rig: a deterministic typed IR, a reproducible per-task RNG substream scheme, an audited candidate-evaluation budget, and a track record of reporting nulls.

- 1.1 The clebran hypothesis
- 1.2 What the experiment found
- 1.3 The reorientation (preview)
- 1.4 Contributions
- 1.5 Outline
- 2.1 DSL program synthesis for ARC
- 2.2 Library learning, compression mining, and MDL
- 2.3 Emergent communication between divergent agents
- 2.4 Learning-to-Theorize and the transfer metric
- 2.5 Multi-agent program synthesis
- 2.6 Curriculum learning and machine teaching
- 3.1 Programs: a linear typed IR
- 3.2 Deterministic interpreter
- 3.3 GP engine
- 3.4 Fitness
- 3.5 The channel and the four payloads
- 3.6 Compression miner
- 3.7 A note on rig-evidence
- 4.1 Sub-hypotheses
- 4.2 Arms
- 4.3 Matched-compute protocol
- 4.4 Subsets
- 4.5 Statistics

- 4.6 What the experiment does not test
- 4.7 Configuration
- 5.1 The Stage-1 first test of H1/H2/H3
- 5.2 The frontier reachability probe
- 5.3 Bandwidth dose-response (H3 on Stage-1)
- 5.4 Recovered-primitive channel (B6/B7)
- 5.5 The central-library control (B5) — the headline arm
- 5.6 Curriculum channel (B8/B9) — even an engaged channel does not beat B5
- 5.7 The teach→test probe and the amortization correction
  - 5.7.1 The teach→test probe (without a cold baseline)
  - 5.7.2 The amortization correction (cold baseline)
  - 5.7.3 The order experiment — the first significant process effect
- 5.8 Vocabulary quality
- 6.1 The reorientation: not what you know, how you learned it
- 6.2 The divergence tax
- 6.3 The role of measurement
- 6.4 Scope: what the result is for, and what it is never for
- 6.5 What this means for related literatures

1. Introduction

The Abstraction and Reasoning Corpus (Chollet, 2019) was designed to measure a property its author calls *skill-acquisition efficiency*: the ability of a system to acquire new competencies quickly, relative to its priors, experience, and generalisation difficulty. In *On the Measure of Intelligence* (Chollet, 2019) the warning is explicit â€” *unlimited priors or unlimited training data let you "buy" skill*, masking a system's actual generalisation power. That framing is what motivates ARC's small training set, its held-out evaluation split, and the deliberate absence of internet-scale supervision. It is also what makes ARC uncomfortable for contemporary systems: a frontier language model scaled with enough pretraining can solve a substantial fraction of the public tasks, but the headline number carries no information about how the model would do if its priors or data were smaller, and therefore tells us little about the acquisition efficiency we say we care about.

This paper is a deliberate step *away* from chasing the headline accuracy. We work in a setting small enough that the acquisition cost is metered rather than guessed: pure-reward evolutionary program synthesis over a hand-designed DSL, on a 30-task frontier subset of ARC-AGI-1, with matched candidate-evaluation budgets audited to 0.1%. The questions we set out to ask concern

whether populations that communicate improve on those that do not; the question we ended up asking concerns *how to measure a learner's acquisition efficiency at all*.

1.1 The clebran hypothesis

The original clebran question (Welsh: *clebran*, to chatter, to gossip) was that multiple populations of DSL programs, each evolving under pure reward on ARC tasks and therefore diverging into genuinely different solution strategies, would solve more problems at matched compute if they were allowed to exchange successful fragments through a constrained DSL channel. The intuition has three parts: (i) that independent reward-driven search will diverge into different abstractions, *because* the search is partial (H1); (ii) that exposing each lineage to a peer's fragments helps it reach solutions it would not reach alone (H2); and (iii) that a *bandwidth-limited* channel preserves better abstractions than an unconstrained library-dump (H3). A later extension (the *recovered-primitive* variant) asked whether sharing subroutines a lineage had *discovered for itself* would beat sharing raw code over a hand-designed vocabulary (the "given versus recovered" comparison, sometimes called Â§11 in the project).

The promise of the clebran design is closely related to two active research literatures. From emergent communication (Lazaridou & Baroni, 2020; Kiela, 2019), it borrows the hypothesis that a bandwidth-limited channel between divergent agents induces compositional, generalisable protocols. From library learning and the DreamCoder/Stitch line (Ellis et al., 2020; Bowers et al., 2023), it borrows the abstraction-discovery mechanism and the *minimum-description-length* pressure that makes discovered subroutines cheap to re-use. The project's specific contribution, in its original framing, was to put those two ideas together on ARC: divergent lineages that gossip their *recovered* subroutines, and the bandwidth of the gossip acts as the abstraction-quality filter.

1.2 What the experiment found

It did not work. The clebran question, as posed, is **falsified** by its own strongest control, and a falsification is the most useful thing a small, well-instrumented study can produce.

The qualitative headline: across nine arms and twenty seeds at matched compute on the frontier subset, the multi-agent arms with a constrained channel are not better than the strongest single-agent baseline â€” and the central-library control (the DreamCoder/Stitch arm) is significantly *better* than the multi-agent channels. The mechanism the clebran design assumed the channel would provide (*sharing of recovered abstractions between divergent populations*) is real but small, and the *cost of splitting compute across populations* is larger than the benefit the channel can pay back. H1 (lineages diverge) is supported; H2 (transfer helps) and H3 (the constraint matters) are null.

The rest of the paper is the data and the consequences. The headline ranking is reported in §5.5; the multi-agent-channel results are in §5.4 (recovered-primitive) and §5.6 (curriculum); the

dose-response and the mechanism behind the H3 null are in §5.3; the teach→test probe and the cold-baseline correction that exposed a hidden confound in the project's first attempt at the probe are in §5.7. The reorientation to acquisition efficiency — which is *why* the headline was hard to see with solve-count as the endpoint — is delivered in §6.1, where the data have earned it. (the constraint matters) are null.

The rest of the paper is the data and the consequences. The headline ranking is reported in §5.5; the multi-agent-channel results are in §5.4 (recovered-primitive) and §5.6 (curriculum); the dose-response and the mechanism behind the H3 null are in §5.3; the teach->test probe and the cold-baseline correction that exposed a hidden confound in the project's first attempt at the probe are in §5.7. The reorientation to acquisition efficiency – which is *why* the headline was hard to see with solve-count as the endpoint – is delivered in §6.1, where the data have earned it.

1.3 The reorientation (preview)

The headline is striking enough to demand a re-reading of the question. The clebran design tried to buy acquisition efficiency by adding *communication* to evolution; the data argue, instead, that the better purchase is to *concentrate* the compute — one agent that sees every task and reuses its own mined abstractions strictly dominates several agents that see task slices and share. The more interesting question, then, is not "does communication help?" but "does the *way a system learns* change how well it can use what it learned?" That is Chollet's re-statement of intelligence as acquisition efficiency, applied at the level of an individual learner. The full argument is delivered in §6.1, where the data have earned it; the contribution list (§1.4) names the two deliverable pieces (the falsification, and a capability audit for what an "acquisition-efficient" learner would need before "how you learned it" can be studied cleanly).

1.4 Contributions

Concretely, the paper contributes:

1. **The clebran design** â€" a typed linear-IR DSL over Hodel's 160 ARC primitives, with a deterministic interpreter, a typed GP engine with phase-1 systematic seeding and adaptive operator selection, a Stitch-style compression miner, a blackboard channel with three constraint regimes and four payload types, and a measurement rig that audits candidate-evaluation budgets to better than 0.3% spread. The implementation is open-source, deterministic, and reproduces 399/400 of Hodel's reference solvers through our interpreter.
2. **A falsified central hypothesis** â€" the multi-agent-with-channel design is dominated by a single agent with a central library at matched compute, with the single significant comparison being **B6 â^' B5 = â^'0.85 (~2.2 se)**. The null is the headline; the baselines are strong enough that a null is trustworthy.
3. **A controlled teachâ†'test protocol and a transfer-vs-reconstruction decomposition** â€" students acquire the teacher's concepts (78% of teaching tasks solved)

but do not transfer them to novel tasks needing the same concepts. This is the L2T transfer-versus- reconstruction gap appearing at the level of whole concepts.

4. **A cost-to-solve (acquisition-efficiency) endpoint for the teaching experiments**, exposing that the effect of teaching lives in evals-to-solve and is invisible under solve count at fixed budget. (Re-scoring the content arms B0–B9 on the same endpoint is a cheap analysis pass left as follow-up.)
5. **A first significant process result** — holding taught *content* fixed and varying only the *order* of acquisition, a curriculum taught foundations-first significantly beats the same content taught foundations-last (paired McNemar $p = 0.021$), because a dependency-violating order halves how much is acquired. It is the study's only significant *process* effect, and it is asymmetric: *don't teach backwards* matters more than teaching in exactly the right order.
6. **A scope claim** — on ARC program synthesis at matched compute, communicating learned *content* does not help and mis-targeted communication hurts, while *how* a system learns — the *process* — is where the substrate shows real structure and is the next question to pursue.
7. **A capability audit** for what an "acquisition-efficient" learner would need before the reorientation can be studied richly.

1.5 Outline

Section 2 positions the work against the relevant literatures (program synthesis for ARC, library learning, emergent communication, L2T/NEO, multi-agent program synthesis, curriculum learning). Section 3 describes the clebran design in detail. Section 4 states the three sub-hypotheses and the arm table, and lays out the match-compute protocol and statistical convention. Section 5 reports the results across H1, H2, H3, the recovered-primitive and curriculum channels, the central-library control, the teachâ†'test probe, and the amortization result. Section 6 discusses the reorientation, the divergence tax, the role of measurement, and the scope. Section 7 lays out limitations. Section 8 concludes.

2. Related work

The clebran design sits at the intersection of four literatures: typed DSL program synthesis for ARC (§2.1), library learning and the DreamCoder/Stitch line (§2.2), emergent communication between divergent agents (§2.3), and the Learning-to-Theorize / Neural Theorizer (L2T/NEO) framework for primitive recovery under partial observability (§2.4). A short note on multi-agent program synthesis (§2.5) and on curriculum / machine-teaching (§2.6) closes the section.

2.1 DSL program synthesis for ARC

The standard reference for typed grid DSLs on ARC is Hodel's `arc-dsl`, which provides roughly 160 typed primitives covering object extraction, geometric transformations, recolouring, and composition. We vendor `dsl.py`, `constants.py`, `arc_types.py`, and (importantly) `solvers.py` — one hand-written solver per training task — into our interpreter. `solvers.py` plays two roles in the project: as a *depth oracle* (the statement count of a solver grades task difficulty and is used to construct depth-graded subsets) and as a *correctness gate* (every reference solver must reproduce its task's grids through *our* interpreter before the task can enter the evolvable subset). 399 of 400 reference solvers reproduce through our interpreter; the one failure (`e40b9e2f`) is order-sensitive upstream and is excluded from the subset.

Two recent LLM-based systems are worth contrasting with the clebran approach because they make the *same* "small system, metered acquisition cost" bet. *ArcMemo* (Ho et al., 2025) proposes an LLM-backed reusable-concept memory for ARC, and is the closest *conceptual* cousin to the central-library (B5) arm — a learner that *accumulates* named concepts over a run. The differences are instructive: ArcMemo's concepts are stored in natural language with LLM-mediated retrieval, ours are recovered executable subroutines in a typed IR with deterministic execution and zero neural components; the clebran design is *intentionally* pre-neural so that acquisition cost is metered, not purchased. The "what to do with a concept" loop is the same in both systems; the "how to find a concept" loop is different (LLM-mediated retrieval in ArcMemo, typed GP search in clebran). The Kaggle ARC Prize 2024–2025 winning entries use ensemble neural approaches that sit orthogonally to the question this paper asks.

The second comparison is the *test-time training* line. Akyürek et al. (2024) is the 2024 ARC Prize second-place paper award; the system performs a small, task-specific network update per task (semi-private 47.5%, public 53% at 8B parameters, 61.9% when ensembled with program synthesis). The TTT line is a direct neural-ARC competitor for the clebran thesis that *test-time compute* (in clebran's case, evolutionary search) is the right place to spend a small system's budget. The antecedent of the TTT-for-ARC line is Sun et al. (2020), which introduced the test-time training framework in a more general setting. The clebran paper and the TTT line make complementary bets about *where* on the curve between pretraining and inference-time compute a small system should sit; we position clebran as the *typed-symbolic* end of that curve.

2.2 Library learning, compression mining, and MDL

The DreamCoder architecture (Ellis et al., 2020) introduced the *wake–sleep* loop for library learning: a neural-guided program synthesiser writes programs from tasks in the wake phase, and a compression step identifies recurring subroutines, adds them to a shared library, and rewrites older programs to use the new vocabulary in the sleep phase. Stitch (Bowers et al., 2023) replaces the expensive compression step with a fast *anti-unification* miner that finds recurring subroutines across programs by over-generalising constants, ranking candidates by the compression ratio they achieve. The clebran library miner is a Stitch-style compression

miner, and the "central library" (B5) arm is a *Stitch-style* learner (no neural recognition model; periodic anti-unification mining; no wake-sleep back-coupling) — pure symbolic search, with library mining run periodically on the population's best programs. We emphasise the *Stitch-style* framing in §5.5 because B5 shares the library mechanism with the multi-agent arms but is *not* a DreamCoder-style wake-sleep learner, and reviewers from the DreamCoder community will distinguish the two.

The MDL pressure on program length is a continuation of a longer lineage: Solomonoff's theory of inductive inference, the Kolmogorov-complexity tradition (Li & Vitányi, *An Introduction to Kolmogorov Complexity and Its Applications*), and the compression-as-cognition line (Schmidhuber, 1997). In the library- learning setting specifically, the MDL pressure was introduced by DreamCoder (Ellis et al., 2020) and inherited by L2T/NEO (Baek et al., 2026). We use the same functional form — `λ_MDL^(-length)` — as the parsimony term in the fitness function (§3.4), with `λ_MDL = 1.05` (the value used in every reported run; see §4.7).

The Bayesian-program-learning antecedent of DreamCoder/Stitch is Tenenbaum (1999) and Goodman et al. (2008), and the broader program-learning lineage runs through RobustFill and the sequence-to-sequence program-synthesis work. L2T/NEO and clebran are both instantiations of this line, but on opposite sides of the neural/symbolic boundary: NEO is probabilistic, operates over latent programs decoded back to observations through a state-grounding constraint, and learns a single codebook by gradient descent on a variational MDL objective; clebran is deterministic, operates over symbolic programs in a typed IR, and *mines* its codebook by anti-unification over best programs. The shared scientific question is whether the recovered vocabulary transfers; their answers differ accordingly.

A note on terminology: in the clebran project, "abstraction" refers specifically to a Stitch-mined subroutine that has been parameterised by its external inputs, invoked as a single program statement, and added to a lineage's private or shared vocabulary. "Fragment" refers to a multi-statement slice of a successful program (the §3.5 channel payload that operates over Hodel's given primitives). "Library" is the shared set of abstractions an arm maintains.

CompressARC (Liao & Gu, 2025) is the direct competitor on the "intelligence as compression" axis. CompressARC is a 76K-parameter neural system with no pretraining and no ARC training-set access that solves 20% of the public evaluation set by minimising a variational description length of the target puzzle. The system took 3rd place at the ARC Prize 2025. Both clebran and CompressARC deliberately sit on the small-system end of the pretraining-vs-inference-time trade-off; both inherit from the MDL / Solomonoff line; the substrate decisions (symbolic program evolution vs neural variational inference) are different. The clebran paper should be read as testing the *symbolic-evolution* instantiation of the compression hypothesis; CompressARC is the *neural- inference* instantiation. A direct empirical comparison would require running clebran's central library on the same evaluation split CompressARC targets, which the clebran paper has not done (see §7).

2.3 Emergent communication between divergent agents

The classical referential-game literature (Lazaridou & Baroni, 2020) asks whether a discrete, bandwidth-limited channel between two agents — one producing, one interpreting — induces a compositional protocol. The standard compositionality metric is *topographic similarity* (Brighton & Kirby, 2006; Lazaridou, Hermann, Tuyls & Baroni, 2018): the Spearman rank correlation between message-space and meaning-space pairwise distances. The clebran project adopts both the protocol (a discrete, bounded channel between populations under independent reward) and the metric (Spearman over arity-1 Grid abstractions). A common finding in the emergent-communication literature is that bandwidth limits the expressivity of the emergent protocol; an *adaptive* learner, however, can also *self-constrain* a too-wide channel by down-weighting operators that rely on the channel. That adaptive mechanism is what makes our H3 (constraint versus unconstrained) test null across all four payload types, and it is the reason the clebran design is at odds with the information-bottleneck story in its naive form.

Two specific results in the emergent-communication literature are directly relevant to the clebran findings. **Bouchacourt & Baroni (2018)** show that successful emergent communication does not entail compositionality: agents establish successful communication by inducing visual representations that align with each other but not with the conceptual properties of the input. This is the same null the clebran paper reports in H3 — bandwidth-limited channels between divergent agents do not, in this substrate, induce better compositionality at matched compute — and the clebran data are a *typed-symbolic re-instantiation* of the Bouchacourt & Baroni finding. **Chaabouni et al. (2020)** show that compositional structure (as measured by various proxies) is decoupled from generalisation, and **Li & Bowling (2019)** show that topographic similarity does not perfectly predict task transfer. The clebran H3 null lives in this empirical envelope: successful communication neither entails compositionality (Bouchacourt & Baroni) nor guarantees generalisation (Chaabouni et al., Li & Bowling), and adding a bandwidth constraint does not move either needle in the clebran substrate.

The clebran H3 null is also in direct conversation with the *information-bottleneck / cultural-evolution theory of language* (Kirby, Tamariz, Smith & Cornish, 2015; Brighton & Kirby, 2006), which argues that compression pressure *does* drive compositional structure in human languages through iterated cultural transmission. The clebran data are a clean empirical counter-example in a typed-symbolic, non-iterated, adaptive-learner substrate: under adaptive operator selection, the constraint and the unconstrained channel produce indistinguishable vocabularies. The H3 null does *not* refute the cultural-evolution account (which involves iterated transmission rather than a single channel between populations) but it does argue that the simple information-bottleneck story is not the whole story even in a non-neural, deterministic setting.

Our setting differs from the referential-game setting in two important ways. First, the agents are *populations* under pure-reward evolution, not neural policies with end-to-end differentiation; the channel is the only inter-agent pathway. Second, the channel operates over a *typed* symbolic substrate (Hodel's primitives or recovered abstractions) rather than over learned discrete

embeddings. The first difference is what makes the project *measurable* on a small budget; the second is what makes the channel's "content" inspectable, a property the neural setting does not have. The clebran EGG-style tooling (Kharitonov, Chaabouni, Bouchacourt & Baroni, 2019) provides the topographic-similarity and discrete-channel primitives the project uses; clebran does not use the EGG framework directly, but borrows the metric and the discrete-channel framing.

2.4 Learning-to-Theorize and the transfer metric

The closest published framework to clebran's actual scientific goal is *Learning to Theorize* (L2T) and its neural implementation NEO (Baek, Lee, Baek, Lee & Ahn, 2026), an ICML 2026 oral. L2T formalises the problem of, given only before→after observation pairs `(x, y)`, recovering the *hidden program* `τ = (z_{i_1}, …, z_{i_K})` — a composition of learned, reusable primitive operations — such that `y = f_τ(x)`. The decisive test in L2T is not reconstruction but *transfer*: infer `τ̂` from a support pair, then execute `τ̂` on a fresh input `x′` and check whether the correct `y′` is produced. The ARC demonstration→test protocol is exactly that transfer test, cast as theory induction.

The clebran design inherits four ideas from the L2T framework, but *three of them are really DreamCoder re-use, not L2T inventions*. We are explicit about this rather than over-attributing, because the intellectual lineage matters: a system can claim to be an instantiation of L2T only if its specific contribution to L2T's hypothesis is identifiable.

- *Theory is an executable program over reusable primitives* — the DreamCoder framing (Ellis et al., 2020; 2021). L2T/NEO inherits this explicitly: NEO is presented as a "Language of Thought" framework, in the same lineage as DreamCoder's wake-sleep Bayesian program induction. clebran adopts the same stance.
- *Adaptive MDL-penalised program length* — also DreamCoder's. DreamCoder 2020/2021 has the same `λ^len(p)` factor. clebran's fitness function uses the same functional form (with `λ_MDL = 1.05`; see §4.7).
- *Primitive recovery under partial observability* — the L2T compositional-OOD claim. NEO recovers primitive-level codes even when some primitives never appear in isolation in training. This is *loosely* connected to H1/H3 (a system under compositional and MDL pressure can disentangle reusable units it never saw alone), but clebran does not run the L2T compositional-OOD split (it is listed as a limitation in §7). The connection is a "shared philosophical root" rather than a "borrowed idea."
- *Transfer, not reconstruction, is the real metric — this* is the L2T contribution. NEO's specific demonstration is the canonical reference, and the clebran teach→test probe (§5.7.1) operationalises it at the level of whole concepts. This is the L2T idea clebran is genuinely building on, and the clebran re-discovery of the transfer/reconstruction gap is a real corroboration of the L2T framing.

The deltas that keep clebran novel against L2T are explicit, and the *substrate delta* is the most important: NEO is *probabilistic* and uses test-time sampling (the paper gives NEO a compute budget at inference and samples hundreds of candidate theories); NEO operates over *latent* programs, decoded back to observations through a state-grounding constraint; NEO's "single codebook" is *learned* by gradient descent on a variational MDL objective. clebran is *deterministic*, operates over *symbolic* programs in a typed IR, and *mines* its codebook by anti-unification over best programs. These are three distinct substrate decisions (probabilistic vs deterministic, latent vs symbolic, learned vs mined) that the substrate delta distributes across. The clean framing is: *NEO and clebran are both instantiations of DreamCoder-style library learning, but on opposite sides of the neural/symbolic boundary; their shared scientific question is whether the recovered vocabulary transfers, and their answers differ accordingly.*

On top of the substrate delta, clebran adds four operational axes that NEO does not have: (a) *multiple deliberately divergent* populations; (b) *pure evolutionary reward* (no differentiable executor, no program supervision); (c) *communication* of recovered abstractions across lineages; (d) a *constrained channel* (the bandwidth-constrained blackboard). The clebran design asks, in L2T terms, whether several theorizers that each theorise the world differently can teach each other their primitives, and whether a bandwidth-limited channel makes those taught primitives *better*. The answer the data give is *no* — but the answer is precise: in this substrate, at matched compute, on this frontier, the taught primitives are no better than those a single agent would mine on its own.

Note on the iterated / single-round axis. NEO is implicitly iterated (gradients are taken across many samples and many training episodes). clebran's reported experiments run *one* round of communication; the iterated form ($\text{curriculum}_1 \rightarrow \text{agents}_2 \rightarrow \text{curriculum}_2 \rightarrow \ldots$) is unaddressed (see §7). The single-round / iterated distinction is a fifth axis in the substrate delta; we flag it as a *limitation* of the clebran result, not as part of the L2T delta.

2.5 Multi-agent program synthesis

Multi-agent formulations of program synthesis appear in the population-based GP literature (Poli et al., 2008), in cooperative co-evolution (Potter & Jong, 1994), and in island models with migration (Skolicki & Jong, 2005). clebran's specific framing — *constrained communication* of *recovered subroutines* between *deliberately diverged* populations — is a novel combination but inherits the island-model concern that migration policy matters as much as migration rate. Our adaptive operator selection (AOS, §3.4) is the clebran answer to that concern: the channel cannot crowd out mutation, because AOS down-weights the fragment operator when it is failing, regardless of how much material the channel supplies. The *empirical* result is that AOS is so effective at this self-regulation that the H3 constraint test is null across all four payload types (§5.3, §5.4, §5.5).

The modern multi-learner / population-based literature has three threads that are in the clebran intellectual neighbourhood. *Population-Based Training* (Jaderberg et al., 2017) is the closest

neural-network-native analogue: a population of learners shares hyperparameters and discovers good schedules through mutation and exploitation. PBT is in the lineage of island models but operates on neural-network hyperparameters rather than program fragments, and its "channel" is a shared parameter store, not a constrained blackboard. *Quality-Diversity* algorithms (Lehman & Stanley, 2011; Cully, 2019) maintain a *diverse* set of solutions rather than converging to the best; clebran's H1 (lineages diverge) is exactly the QD premise applied to a population-of-populations setting, and the clebran use of `Random(f"{seed}:{task_id}")` to guarantee seed-level divergence is the simplest possible QD-style *novelty* mechanism. *Federated learning* (McMahan et al., 2017) is the modern distributed-learning analogue of "many learners share a parameter server"; not a perfect fit (clebran's channel is a blackboard, not parameter averaging), but the structural parallel is close enough to deserve a sentence. None of these threads combine (a) *deliberate divergence* across populations, (b) *constrained* symbolic communication, and (c) *typed* symbolic payloads — the clebran combination.

2.6 Curriculum learning and machine teaching

The *curriculum channel* arm of clebran (§3.5.4) is a deliberate synthesis of two threads. *Machine teaching* (Zhu, 2015; Liu et al., 2017; Garcia-Piqueras & Hernández-Orallo, 2021) inverts the learning problem: given a target concept, find the smallest teaching set that induces it. The closest compositional variant — *Optimal Teaching Curricula with Compositional Simplicity Priors* (Garcia-Piqueras & Hernández-Orallo, 2021) — uses an MDL-style prior to pick the teaching set, which is the same prior that clebran's fitness function uses to pick among programs. *Automatic curriculum learning* in RL (Portelas et al., 2020; Narvekar et al., 2020) — teacher–student curricula, learning-progress-driven task selection, regret-based environment design (PAIRED, ACCEL; Dennis et al., 2020; Parker-Holder et al., 2022) — provides the curriculum-as-policy framing. POET (Wang et al., 2019) and the asymmetric self-play line (Sukhbaatar et al., 2018) are the closest existing examples of *communication through tasks*: Alice proposes goals, Bob solves them, and competence compounds. clebran's specific addition is the *bandwidth-constrained multi-teacher* setting: many divergent teachers, each exporting a *task sequence* (not a single goal, not a gradient, not code) over a constrained channel to many divergent students. We are not aware of prior work combining all four axes (multi-agent divergence, curriculum-as-payload, constrained channel, program-synthesis substrate).

The teach→test result of §5.7.1 — students learn taught concepts (70–78% of synthetic teaching tasks solved) but do not transfer them to novel tasks needing them — sits inside a larger literature on negative transfer and continual learning. The findings are consistent with the catastrophic-forgetting and plasticity-loss observations in continual learning (Kirkpatrick et al., 2017; Schwarz et al., 2021): a learned vocabulary can bias later search in ways that hurt tasks the vocabulary does not cover. The negative-transfer phenomenon is the broader topic; the *multitask-learning* line (Caruana, 1997) and the *task-affinity* line (Standley et al., 2020) provide the formal machinery. The clebran 8b / "irrelevant teaching hurts" result is a negative-transfer result in this lineage, and the *compositional-generalisation* benchmarks — SCAN (Lake

& Baroni, 2018), COGS (Kim & Linzen, 2020), gSCAN (Ruis et al., 2020) — provide the controlled-distribution probe the clebran paper has not run (see §7).

3. The clebran design

The clebran system has six major components: a typed intermediate representation for programs (§3.1), a deterministic interpreter with size caps and a structural grid validator (§3.2), a typed GP engine with phase-1 systematic seeding and adaptive operator selection (§3.3), a foreground/background-balanced fitness function with MDL pressure (§3.4), a per-task lineage model with a private library and a shared channel (§3.5), and a Stitch-style compression miner that recovers abstractions from a lineage's best programs (§3.6). The section is a technical reference; the experimental protocol sits in §4.

3.1 Programs: a linear typed IR

Programs are *linear SSA-style register programs* over Hodel's 160 typed grid primitives:

```
x0 = fn(...); x1 = fn(x0, ...); ...; return x_n
```

The same shape as Hodel's hand-written solvers, which means the reference solvers parse *losslessly* into our IR — that losslessness is what allows the depth-graded subset selection (§4.1) and the correctness gate to work on real data.

Typing is a *nominal* typing of primitives plus a *union* flow-through rule: when a primitive returns a union type, every downstream use of the result is typed with the union. The crucial case is the `vmirror: Piece → Piece` signature, which the union rule allows to be fed into a `Grid` slot at the next call (because the Grid slot accepts `Grid | Piece`). Without the union rule, typed mutation could never build the `vmirror(I)` → crop → paste compositions that several reference solvers require, and a substantial fraction of the depth-graded subset would be unreachable by the GP engine. The union-flow bug was the first diagnosed failure in passing the Stage-1 acceptance gate (§3.7).

The IR is also used to encode the *abstraction call* — a single program statement `("abstr", name, *args)` — that invokes a recovered subroutine at run time, inlines its body through `interpreter._run_body`, and costs one unit of program length for MDL purposes. The abstraction's signature is computed from the dependencies of the duplicated slice and registered in a per-lineage table; the typing machinery looks up the table when typechecking the call site.

3.2 Deterministic interpreter

The interpreter is a small, exception-guarded evaluator with two size caps (a max-register count, set to keep linear programs from exploding; a max-statement count) and a structural grid

validator that returns the *first row* of the result and a structural hash representing the grid's shape, plus a content hash of the cells, in O(rows + width) time. The validator was the dominant cost in the interpreter before profiling: a per-cell isinstance check ran ~80M calls in a 32-second profile window. The O(rows + width) validator preserves behaviour (the 399/400 reproduction rate is unchanged) and brings throughput from ~2.8k evals/second/process to ~8.3k.

A *length-guarded fitness cache* dedupes candidate evaluations whose programs are length ≥ 4 by a hash of the program string. Every deduped evaluation still counts as a candidate evaluation; the cache only removes redundant *work*, not redundant *budget*. The verification protocol (§4.2) checks that budgets and search outcomes are byte-identical with and without the cache enabled.

The interpreter exposes a `run_trace` function that returns every register value as an ARC grid, used by the solver-card HTML report to render a step-by-step execution trace of every evolved program. The trace is also used internally by the compression miner to identify subroutines that recur across programs (§3.6).

3.3 GP engine

The engine is a typed genetic programming system operating on the linear IR. The operators are:

- **Point mutation** — flip a head to a compatible primitive, or perturb a constant.
- **Argument mutation** — pick a different source register or constant for an existing call.
- **Grow mutation** — insert a new statement at a typed position and rewire downstream uses to consume its output. On the linear IR, this is the analogue of "subtree replacement" in tree-GP.
- **Two-segment crossover** — swap a contiguous slice of statements between two programs, with a repair pass that re-types the splice.
- **Fragment splice** — insert a received fragment (from the channel) as a contiguous slice, with hole-filling by typed mutation.
- **Depth-1 / depth-2 systematic enumeration** — exhaustively enumerate programs of bounded length and a fixed head, used at the *phase-1 seeding* step.

Phase-1 systematic seeding. The first ~5% of the budget is spent on a deterministic closure of the depth-≤2 bucket: depth-1 enumeration over typed constants; depth-2 skeleton sweep; a signature-niched population; and constant-polish (coordinate descent plus bounded 2-D joint sweeps for two-coordinate needles like crop offset + size) on the top stubs *and* compositions around the top families. The seeding is identical across arms, so any channel-driven gain must come from the evolutionary phase, not from the seeding phase. We report *evolution-phase solves* separately from seeding solves for exactly this reason.

Adaptive operator selection. Each operator's share of the offspring production is set by its recent child-≥-parent success rate, with floors. The fragment splice is the operator that adaptive

selection most often down-weights: in the K×M dose-response (§5.3), fragment splice succeeds ~2% of the time vs ~36% for mutation, so AOS holds it near a 3% share *regardless of how much material the channel supplies*. AOS is the mechanism that makes the H3 constraint test null: an adaptive learner self-constrains a failing channel.

Per-task RNG substreams. Each task's trajectory is driven by `Random(f"{seed}:{task_id}")`, so a task behaves bit-identically in isolation and interleaved with the other 29. The isolation *predicts* the full-run behaviour, which is the property that makes the per-task debugging loop tractable.

Progress-weighted task scheduling. Generations are allocated by proximity-to-solution instead of round-robin. Without this, near-miss tasks were starved in the budget tail: a task that is 1 cell from solving would not get the generations that would let a polish mutation hit it, because the schedule moved on to a task further from its own solution.

Hall-of-fame restarts. Stagnation restarts explore fresh space but re-inject the best individual per skeleton signature, so a restart cannot destroy polishable partial-credit stubs. Without the hall-of-fame injection, the B0 Stage-1 gate fails (several tasks have a unique 2-cell fragment that only appears in a single mutation's neighbourhood; a clean restart loses the search neighbourhood entirely).

3.4 Fitness

The fitness function is *pure outcome reward*: the fraction of demo pairs a candidate program solves exactly, *tie-broken by per-cell accuracy on the foreground and on the background separately*. The background/foreground split is critical: on mostly-background targets, plain per-cell accuracy lets a `canvas(ZERO, 3×3)` — a blank grid of the right size — outscore every content-bearing program. The split is implemented by counting, per target, the *background colour* (the most common cell value, or the cell value at the corners for framed tasks) and scoring the foreground and background cell accuracies separately. The reward is then:

```
reward(p) =
  foreground_acc(p) * foreground_weight
  + background_acc(p) * background_weight
  - foreground_penalty * (1 - foreground_present(p))
```

with weights chosen so that a fully-foreground program with background accuracy 0 still loses to a fully-correct program, and a fully-background "all-blank" program loses to any program that produces a non-trivial foreground.

The reward is then scaled by NEO's MDL factor:

```
final(p) = reward(p) * (λ_MDL ** -len(p))
```

with `λ_MDL = 1.05` (the value used in every reported run; see the configuration table in §4.7 for the code default). The factor penalises long programs; combined with the foreground/background split, it is what makes the population converge to the *shortest correct* program rather than the longest-fitting one.

The fitness function is *test-exact-aware*: a candidate that solves all demonstration pairs is then applied to the test input(s) and the resulting grids are recorded as a `test_exact` flag. The transfer-vs-reconstruction gap (§5.7.1) is the difference between solving all demos (reconstruction) and reproducing the test grid exactly (transfer). Both flags are logged for every candidate.

3.5 The channel and the four payloads

The channel is a blackboard shared by all lineages of a single arm. A lineage publishes messages to the blackboard at a periodic publish cadence (every K epochs), and reads messages that other lineages have published since its last read. The constraint regime imposes a *bandwidth* (K, the maximum message size in statements or in body length) and a *rate* (M, the maximum number of messages per lineage per epoch). Three constraint regimes are supported: **CH-NONE** (no channel at all; the B1 control), **CH-CONSTRAINED**, and **CH-UNCONSTRAINED** (effectively M = ∞). Four payload types are supported.

Fragment payload. A *fragment* is a multi-statement slice of a successful program, parameterised by the external inputs it reads. A lineage publishes its best-performing fragments; receivers splice them in as a contiguous block with hole-filling by typed mutation. This is the original §3 payload, used by B2/B3/B4.

Signal payload. A *signal* is a discrete identifier that names a target difficulty bucket, used as a routing hint rather than a code fragment. Signal payloads are the B4 arm; they test the *abstract-communication* hypothesis (H3 variant) — that a *less* informative channel might nonetheless be useful as a routing signal — and have been used in the Stage-4 B0–B4 sweep.

Recovered-primitive payload. A *recovered abstraction* is a subroutine that the publishing lineage has mined from its own solved programs using the compression miner (§3.6), parameterised by its external inputs, and registered in the shared abstraction table. A *received* abstraction is installed into the receiving lineage's local library as a new primitive with the mined signature. The reuse rate is the fraction of solves that contain at least one received abstraction; the *received-used* metric is the average number of received abstractions used per solve (the NEO-style "used a primitive it was handed and never derived" signal). This is the B6/B7 arm and is the §11 scientifically important variant.

Curriculum payload. A *curriculum* is an ordered sequence of synthetic tasks generated by the publishing lineage from its own DSL: a program that uses a target abstraction is sampled, run on random input grids through the interpreter, and the resulting (input → output) demonstration pairs form a teaching task. The teaching tasks are ordered by the prerequisite

structure of the abstractions they were synthesised from (a `score` field in the task record). A *curriculum-aware* receiver carries a library-persistent state and gates deeper tasks behind their prerequisites being solved. The reuse rate is not applicable to curriculum payloads (no fragment or abstraction is transmitted); the engagement metric is *aux_received* (teaching tasks received per lineage) and *aux_solved* (of those, how many the student solved). This is the B8/B9 arm, the §7b hypothesis.

3.6 Compression miner

The compression miner is a Stitch-style duplicated-slice miner. Given a set of programs (typically, the top-k by fitness from a lineage's hall of fame), it finds dependency-closed subroutines that recur across at least two programs, with the same primitives, the same wiring, and the same constants up to a substitution. The candidate subroutines are parameterised by the inputs they read that come from outside the slice; the resulting `Abstraction` is registered in the lineage's library as a new primitive with a typed signature. The rank order is by *compression ratio*: the number of statements removed from the rewritten programs, minus a body-length penalty.

Configuration. B5 (the central-library control, §5.5) calls `mine_own_abstractions(max_new=8, min_size=2, max_size=4)` every `mine_every=5` epochs (`clebran/baselines/b0.py`). The underlying `mine_compressing` requires `min_occurrences=2` — a subroutine must appear in at least two of the lineage's best programs to be considered. The 5-epoch period is unusually frequent for a library- learning system; it is the choice that makes B5 grow its library aggressively. A sensitivity check (varying `mine_every` from 1 to 20) is a natural follow-up; the paper uses `mine_every=5` because the comparison with the multi-agent arms requires the central- library mechanism to have a comparable number of mine calls per run.

On Hodel's 400 reference solvers, the miner recovers meaningful subroutines unsupervised: `ofcolor(I, leastcolor(I))`, `argmax( objects(I, …), size)`, `remove(ZERO, palette(I))`, and so on. The topographic similarity of the resulting vocabulary is 0.53 over arity-1 Grid abstractions — a real compositional signal. The miner is also the component that B5 uses: B5 is essentially a single population that periodically runs the miner over its hall of fame and adds the top compression-ratio abstractions to its library. The fact that B5 dominates the multi-agent arms is the headline result of §5.5.

3.7 A note on rig-evidence

The spec's Stage-1 acceptance gate — B0 solves all 10 near-atomic tasks on *every* seed — forced a series of mechanisms (union-type flow-through, foreground/background-balanced reward, hall-of-fame restarts, per-task RNG substreams, progress-weighted scheduling, adaptive operator selection) that are now part of the rig described in §3.1–§3.6. The Stage-1 debugging trail is documented in detail in §6.3, where the "what it took to make the rig reliable" story

belongs. Every fix is identical across arms, so the H2/H3 comparison is not biased by which fix landed first.

4. Hypotheses, baselines, and experimental protocol

The clebran design tests three sub-hypotheses of the central claim plus one extension. Section 4.1 states them precisely; §4.2 maps each to one or more arms; §4.3 specifies the match-compute protocol; §4.4 describes the subsets; §4.5 the statistical convention.

4.1 Sub-hypotheses

H1 (divergence). Independently-seeded lineages under pure reward discover genuinely different abstractions for the same task distribution. *Measurable:* pairwise Jaccard distance over primitive usage; fraction of tasks solved by exactly one lineage in the multi-lineage arms.

H2 (transfer helps). Sharing successful subprograms between lineages increases total solve count at matched compute, relative to not sharing. *Measurable:* the B2 (fragment channel) – B1 (isolated) comparison, and analogous comparisons for B6, B8, B9.

H3 (the constraint matters). A bandwidth-limited channel preserves better or more reusable abstractions than an unconstrained "dump the whole library" channel. *Measurable:* the B2 – B3 comparison, the B6 – B7 comparison, the B8 – B9 comparison, and the K×M dose-response ladder.

§11 (recovered > given). Sharing subroutines a lineage has *mined for itself* outperforms sharing raw fragments over Hodel's given primitives. *Measurable:* the B6 (recovered) – B2 (given) comparison.

§7b (curriculum-as-payload). A teaching set — a sequence of synthetic tasks that *show* what the sender learned — is a useful alternative to sharing code. *Measurable:* the B8 (curriculum constrained) – B1 comparison; the B8 vs B2/B6 comparisons on identical compute.

A *central* comparison that is not strictly part of the original hypothesis set but emerged as decisive is the B5 vs B6 comparison: B5 is a single population that grows a central library by mining its own solutions — the DreamCoder/Stitch control. If a multi-agent-with-channel design is to win on this substrate, it must beat the single agent that has the same library mechanism without the divergence tax.

4.2 Arms

arm	what it is	tests
B0	single population, all compute	reference line
B1	N isolated lineages, no channel (CH-NONE)	H1 baseline
B2	N lineages + CH-CONSTRAINED fragment	H2, H3 (given)
B3	N lineages + CH-UNCONSTRAINED fragment	H3 (given)
B4	N lineages + CH-CONSTRAINED signal	abstract comms variant
B5	single population + central library	*central-library control*
B6	N lineages + CH-CONSTRAINED recovered-primitive	H2, H3 (§11)
B7	N lineages + CH-UNCONSTRAINED recovered-primitive	H3 (§11)
B8	N lineages + CH-CONSTRAINED curriculum	§7b (curriculum)
B9	N lineages + CH-UNCONSTRAINED curriculum	§7b (curriculum)

Multi-lineage arms use N = 3 lineages and 20% primitive-pool dropout per lineage, ensuring divergence at the seed level. All arms are run at matched candidate-evaluation budgets (§4.3) on the same subset and the same seeds.

4.3 Matched-compute protocol

The compute unit is the *candidate evaluation*: every call to `fitness.Evaluator` against a specific program and task, whether the program is novel, deduped by the length-guarded cache, or re-evaluated after a mutation, counts as one. The total budget per run is the per-arm budget (`b` evals); the budget is split equally between the N lineages of a multi-lineage arm (so the per-lineage budget is `b/N` and the total is the same as the single-population arm's).

A run is logged to `logs/<arm>_seed<...>.jsonl` as a `start` record (the configuration), `solve` records (task, evals consumed, program JSON, test-exact flag), `epoch` records (lineage state summaries every K generations), and a final `summary` (solved sets, per-lineage breakdowns, primitive-usage divergence, reuse rate, messages published). Compute audits compare the *recorded evaluations* of the multi-lineage arms to the budget and report the per-arm spread. Audited spreads are ≤ 0.27% across all multi-arm runs reported here; for the headline 20-seed frontier runs the spread is 0.10%. Wall-clock is never used as a comparator; the compute is metered inside `fitness.Evaluator`.

4.4 Subsets

Two subsets are used.

Stage-1 subset. 30 tasks generated deterministically by the recipe in spec §4.1: 10 tasks at depth 1, 15 at depth 2, 5 at depth 3, stratified by reasoning type (object-extraction, geometry, recolouring, composition), each gated by successful reproduction through our interpreter. Committed as `data/stage1_subset.json`. This is the subset used in the Stage-4 B0–B4 sweep and the K×M dose-response.

Frontier subset. 25 D2 + 5 D3 tasks, depth-graded as the Stage-1 subset, but chosen by the additional criterion that the task is *not* solved by phase-1 systematic seeding on a B0 pilot run. This is the subset used in the recovered-primitive and curriculum channel experiments and the B5 central-library control. Committed as `data/frontier_subset.json`. The frontier subset is the one the paper reports, because the Stage-1 subset's solvable frontier is shallow enough that phase-1 closes it without communication — a feature that would mask any channel-driven gain.

The depth proxy is the number of DSL primitive calls in Hodel's reference solver, computed by `solver_parse.parse_all_solvers`. D1 is depth ≤ 2 (1–2 statements), D2 is depth 3–5, D3 is depth 6–10. The pilot study that defined the frontier found that genuine evolution-phase solves occur only in D2 at 2M evals (seeds 1–2) — D3 is beyond reach and would just add a floor of zeros, so the frontier is depth-2-dominated.

4.5 Statistics

For each arm, we report the mean and standard deviation of *evolution-phase solves* across seeds. Evolution-phase solves are the tasks reached by the evolutionary loop *after* the phase-1 seeding is complete, so the number reflects only what search (and communication) contributed. Phase-1 is identical across arms, so including it would only add a constant.

The standard error of a mean difference is the pooled standard error: `sqrt(sd_a^2/n_a + sd_b^2/n_b)` for means of equal-sized samples. We report a comparison as *significant* if the absolute difference is at least 2.0 standard errors. We deliberately do *not* use a t-test or a binomial test on the headline comparisons because the solve counts are small (1–3 evolution solves/seed) and the per-seed distributions are not normal. The standard error is a *descriptive* statistic for the magnitude of the effect, not a hypothesis-test p-value.

For the cost-to-solve comparisons in §5.7.1, we use a permutation test (10 000 to 20 000 resamples) on the *paired* differences between arms, restricted to the (target, seed) pairs that were solved by both arms under comparison. The permutation test does not require normality and is the right tool for the small-n, paired, censored case. The permutation p-values are reported alongside the observed differences.

4.6 What the experiment does *not* test

- **Other substrates.** The clebran design is scoped to ARC program synthesis over a hand-designed DSL. It does not test the channel hypothesis on language-model agents, robotic

manipulation, or other multi-agent settings. The transfer of the negative result to those substrates is a question for future work.

- **The eval split.** All reported results are on the training split. The 400-task eval split is reserved for a follow-up generalisation test; it has not been touched in the reported experiments, and we flag it as a limitation in §7.
- **Compositional OOD.** The L2T compositional-OOD and length-OOD splits (Baek et al., 2026) are an interesting generalisation probe; we have not run them. They are listed in the limitations as an obvious next measurement.
- **The loop.** A single round of sharing has been run; the iterated form (curriculum$_1$ → agents$_2$ → curriculum$_2$ → …) has not. The loop is the actual open-endedness question, and is unaddressed here.

4.7 Configuration

A competent reader should be able to reproduce the headline from the paper alone. The table below names every hyperparameter the code uses in the reported runs. Sources: `clebran/fitness.py`, `clebran/lineage.py`, `clebran/channel.py`, `clebran/orchestrator.py`, `clebran/dsl/abstraction.py`, `clebran/baselines/b0.py`. Items marked **(stated)** are reproduced from the paper text; items marked **(not stated)** were not in the paper before this revision and were added to make the experiment auditable from the paper alone.

hyperparameter	value	where in code	paper status
`λ_MDL` (MDL parsimony factor)	**1.05**	`fitness.py:38`, `lineage.py:103`, `orchestrator.py:32`	(was 1.1; corrected)
`K` (max fragment size, statements)	4	`channel.py:86`	(now stated)
`M` (max messages/lineage/epoch)	3	`channel.py:87`	(now stated)
N (lineages per arm)	3	`orchestrator.py:154`	(stated)
`drop_frac` (primitive-pool dropout per lineage)	0.20	`orchestrator.py:31`	(stated)
`pop_size` (population per task)	100	`lineage.py:97`, `orchestrator.py:156`	(now stated)
`tournament_k`	3	`lineage.py:98`	(now stated)
`elite`	5	`lineage.py:99`	(now stated)
`p_crossover`	0.30	`lineage.py:100`	(now stated)
`p_fragment` (prob. of fragment-splice vs other ops)	0.15	`lineage.py:101`, `b0.py:29`	(now stated)
`max_library` (per-lineage fragment library)	50	`lineage.py:102`	(now stated)
AOS decay (op_n/op_w recency)	0.99	`lineage.py:84` `op_credit(decay=0.99)`	(now stated)
AOS op-weight floor	0.05	`lineage.py:81`	(now stated)
Hall-of-fame size cap per task	10	`lineage.py:76`	(now stated)
Fitness cache: threshold length	4	`fitness.py:62`	(now stated)
Fitness cache: cap (entries)	300 000	`fitness.py:53`	(now stated)
`mine_every` (B5 compression-miner period, epochs)	5	`b0.py:32, 84`	(now stated)
`max_new` (B5 abstractions per mine call)	8	`lineage.py:673`	(now stated)
`min_size` / `max_size` (B5 abstraction body bounds)	2, 4	`lineage.py:673–674`	(now stated)
`min_occurrences` (compression miner duplicate threshold)	2	`abstraction.py:274`	(now stated)

hyperparameter	value	where in code	paper status
Curriculum payload `n_pairs` per sub-abstraction	4	`orchestrator.py:78`	(now stated)
Curriculum aux-task per-lineage cap	12	`orchestrator.py:22`	(now stated)
Budget: Stage-4 sweep	1 000 000	experiment README	(stated)
Budget: frontier 8-seed B0/B1/B2/B6/B7	3 000 000	experiment README	(stated)
Budget: frontier 20-seed B0/B1/B2/B5/B6/B7/B8/B9	4 000 000	experiment README	(stated)
Seeds: Stage-1 5-arm sweep	5 (1–5)	experiment README	(stated)
Seeds: K×M dose-response	3	`dose_response.py` default	(stated)
Seeds: frontier 8-seed	8	experiment README	(stated)
Seeds: frontier 20-seed	20	experiment README	(stated)
1-failure case in 399/400 reproduction	`e40b9e2f`	`tests/test_stage0.py:30-31`	(now stated) — order-sensitive in Hodel's solver

The Stage-4 sweep is committed at `40067c7`; the other experiments do not currently state their commit hash in the experiment READMEs, and the consolidated audit table is deferred to the per-experiment analyses and to `git log experiments/`.

Note on the use of "significant". Throughout the paper, "significant" is used *colloquially* to mean "above the 2-SE descriptive bar" (the convention in §4.5), *not* in the formal hypothesis-testing sense. The paper does not use t-tests, p-values, or confidence intervals. The 2-SE bar is a *descriptive* heuristic for effect-magnitude; a comparison that does not meet it is not *proven null*, only *under-resolved at the current seed count*.

5. Results

We report the experimental results in the order in which they were produced. Section 5.1 reports the Stage-1 first test of H1/H2/H3 on the Stage-1 subset. Section 5.2 reports the B0 reachability probe on the frontier subset, which established the frontier as the right place to look. Section 5.3 reports the bandwidth dose-response. Section 5.4 reports the recovered- primitive channel result and its power check. **Section 5.5 reports the central-library control (B5) — the headline arm. Section 5.6 reports the curriculum channel, framed as "B5 is so**

good that even the curriculum channel — whose mechanism does work — cannot beat it." Section 5.7 reports the teach→test probe and the amortization result that corrected it. Section 5.8 reports the vocabulary-quality metrics.

5.1 The Stage-1 first test of H1/H2/H3

The first multi-arm run was the Stage-4 sweep on the Stage-1 subset, 5 arms × 5 seeds × 1M evals, audited to 0.27% compute spread. The headlines are:

- **H1 supported.** Pairwise Jaccard distance over primitive usage, averaged over the multi-lineage arms and 5 seeds, is 0.57 ± 0.10. The minimum Jaccard across the 25 lineage-pairs is 0.41; the maximum is 0.78. Tasks solved by exactly one lineage appear in 4/5 seeds. The lineages *do* diverge into different solution strategies.
- **H2 null.** The fragment channel (B2) and the unconstrained fragment channel (B3) are statistically indistinguishable from the isolated baseline (B1). B2 = B1 = 10.4/30, B0 = 10.6/30. Channel reuse on Stage-1 is at most 0.17 (B3) — the channel barely engages, because the Stage-1 solvable frontier is shallow enough that phase-1 systematic seeding closes it without communication.
- **H3 directional, not significant.** B3 is the *worst* arm at the highest reuse (9.80/30), directionally consistent with the unconstrained-sharing-hurts story. The pooled standard error is large enough that the gap is not significant (B2 − B3 = +0.60, se 0.32); the result is suggestive, not confirmatory.

[exp: 2026-07-20_stage4_b0-b4_matched_compute]

The Stage-1 result is the first indication that the clebran hypothesis, as posed, is not the cheap win the design assumed. A null at this stage is *structural*, not a budget artifact: the solvable frontier is closed by phase-1, so the channel has nothing to add.

5.2 The frontier reachability probe

A two-seed B0 pilot at 2M evals on the frontier subset established the regime in which a channel effect could plausibly appear. Three results.

1. **Phase tagging is correct.** The one D3 solve in the pilot was a short accidental depth-2 solution, correctly tagged phase-1. Every genuine evolution-phase solve was D2. The evolution phase *only* reaches D2 at 2M evals.
2. **No D3 evolution solves.** Zero D3 evolution solves across 2 seeds at 2M evals. D3 (depth 6–10) is beyond reach and would just add a floor of zeros, so the frontier is *composition-reweighted* to 25 D2 + 5 D3.
3. **Counts are small.** 1–2 evolution solves per seed per 2M evals. The treatment signal, if any, is small; the budget and seed count must be sized for it.

A pilot of B1 vs B2 on the frontier at 3M evals, 2 seeds, confirmed the frontier rebuild: B2 mean evolution-solves 2.0 vs B1 1.5, and channel reuse *jumped* from 0.06 to 0.67–1.00. The mechanism H2 depends on is now active; the frontier rebuild worked. Held for the main sweep on the same compute decision as the next item.

[exp: 2026-07-20_frontier_pilot]

5.3 Bandwidth dose-response (H3 on Stage-1)

A K×M bandwidth ladder on the Stage-1 subset (`none` → K2M1 → K4M3 (=B2) → K8M10 → `uncon` (=B3)), 3 seeds, 1M evals, tests the dose-response shape and the hypothesized crowding mechanism. The result is *flat* and mechanistically informative. We report **evolution-phase solves** here (not total solves), because H2 and H3 are *defined* on evolution-phase solves — the only place the channel can act.

config	evo-solved/30	reuse	frag-share	frag-succ	mut-succ
none	2.00 ± 1.00	0.00	0.03	0.02	0.36
K2M1	2.00 ± 1.00	0.04	0.03	0.02	0.36
K4M3	2.33 ± 0.58	0.05	0.03	0.02	0.36
K8M10	2.00 ± 1.00	0.16	0.03	0.02	0.36
uncon	1.67 ± 0.58	0.17	0.03	0.02	0.36

Three things to read off the table. First, *no dose-response*: the evolution-only solve rate is flat (1.67–2.33) across the whole ladder, all overlapping in noise; the unconstrained endpoint is *nominally* the worst (1.67) but the gap to the constrained middle (2.33) is within 1 SE. Second, the *crowding mechanism does not occur*: `frag-share` is pinned at 0.03 *regardless of bandwidth* — identical even for `none`, which has no channel at all. Bandwidth changes *which* fragments are used (reuse rises 0 → 0.17) but not *how much* fragment-splicing happens. Third, the reason is adaptive operator selection: AOS weights the fragment operator by its ~2% success vs ~36% for mutation, and floors it near 3% share no matter how much material arrives. An adaptive learner is robust to unconstrained sharing because it self-constrains; the constraint stops mattering.

[exp: 2026-07-20_b3_dose_response]

A caveat on the H3 test. Adaptive operator selection is a *design choice* in the rig (added during Stage 1, §6.3) — not a measured property of the learner. The H3 question (does a bandwidth constraint preserve better abstractions than flooding?) is in this design *partially confounded with AOS*: AOS regulates the channel's intake regardless of bandwidth, so H3 may be *untestable* in this substrate, not just null. The dose-response is a *measurement* of the AOS regime, not a clean test of the H3 mechanism. The frontier H3 tests (§5.4) are run on the same AOS regime but in a regime where reuse is much higher (0.67–1.00) and the channel is more

engaged; the H3 null persists there too (B6 ≈ B7, §5.5), consistent with the same AOS explanation.

5.4 Recovered-primitive channel (B6/B7)

The first payoff measurement on the frontier: 8 seeds, 3M evals, B0/B1/B2/B6/B7. The recovered-primitive channel is the §11 scientifically important variant: lineages abstract their *own* subroutines and communicate those, rather than fragments over Hodel's given primitives.

arm	what	evo-solved /30	reuse
B0	single population	**2.12 ± 1.13**	—
B1	3 isolated	1.50 ± 0.53	0
B2	+ constrained fragment	1.38 ± 0.92	0.66
B6	+ constrained recovered-primitive	1.88 ± 0.99	0.67
B7	+ unconstrained recovered-primitive	2.12 ± 1.25	0.67

comparison	Δ (se)	reads as
B6 – B1	+0.38 (0.40)	recovered-primitive transfer helps
B6 – B2	+0.50 (0.48)	recovered beats given-primitive transfer
B6 – B7	−0.25 (0.56)	constraint does **not** help (H3)
B2 – B1	−0.12 (0.38)	fragment transfer doesn't help

The mechanism is real. The *received-used* metric — the average number of received primitives used per solve — is 1.21 for B6 and 1.42 for B7: lineages solve tasks using primitives they were *handed and never derived*. (The metric is a continuous count of received abstractions used per solve, not a binary flag.) A concrete example: a lineage discovered a 4-statement mirror-tiling subroutine `vmirror → hconcat → hmirror → vconcat` and used it as a single named call to solve task `62c24649` with a length-2 program.

The result is directionally the §11 hypothesis — recovered > given — and the divergence tax is paid back: B7 = 2.12 = B0. Nothing is *significant* (counts are 1–2 evolution solves/seed; se's are 0.4–0.5 and swamp the effects), and the comparison is honestly underpowered.

[exp: 2026-07-21_frontier_recovered]

A power check — 20 seeds (8 → 20), 4M evals (3M → 4M), with the new compression miner and the 3× faster engine — gives:

> ***Note on the cross-run comparison.*** *The 20-seed run uses the new compression miner and the ~3× faster engine. The 8-seed and 20-seed B0 measurements (2.12 vs 2.25) are*

> *different measurements, not the same code re-run. The paper treats the 20-seed run as the canonical measurement for the §5.5 headline; the 8-seed run is reported here for the "consistent with its own 8-seed subset" claim. The 20-seed run is not a replication of the 8-seed run — it is the same code on the same tasks with a faster engine and a better miner. Cross-run consistency is therefore not independent evidence of the effect; the proper replication would be 20 new seeds on the 8-seed code, which has not been run.*

arm	what	evo-solved /30	reuse	topsim
B0	single population	**2.25 ± 1.07**	—	—
B1	isolated	1.45 ± 0.94	0	—
B2	fragment channel	1.60 ± 0.99	0.46	—
B6	recovered-primitive (constrained)	1.80 ± 1.06	0.59	0.14
B7	recovered-primitive (unconstrained)	1.85 ± 1.04	0.64	0.15

comparison	Δ (se)	8-seed	verdict
B6 – B1	+0.35 (0.32)	+0.38	directional, **not significant**
B6 – B2	+0.20 (0.32)	+0.50	shrunken, **not significant**
B6 – B7	−0.05 (0.33)	−0.25	**null** (H3)
B2 – B1	+0.15 (0.31)	−0.12	directional, not significant

The ordering is identical across the two runs:

> *B0 ≥ B7 ≈ B6 > B2 > B1.*

Recovered-primitive channels (B6/B7) are consistently the best channel variants. But the 8-seed run's "B7 matches B0" (2.12 = 2.12) does not survive more power: at 20 seeds B0 (2.25) is clearly ahead of B7 (1.85). The divergence tax is *partly* paid back, not fully. More power did not produce significance — it confirmed the effect is small. The +0.50 headline from the 8-seed run was partly noise.

[exp: 2026-07-22_frontier_recovered_scaled]

5.5 The central-library control (B5) — the headline arm

B5 is a single population that periodically runs the compression miner over its hall of fame, installing the top compression-ratio abstractions as new primitives in the population's library. Concretely, B5 calls `mine_own_abstractions(max_new=8, min_size=2, max_size=4)` every 5 epochs, with the underlying miner requiring `min_occurrences=2` (a subroutine must appear in

at least two of the lineage's best programs to be considered). The B5 arm is run at the *identical* config to the scaled B0/B6/B7 sweep so all arms are directly comparable.

> ***A note on framing.*** *B5 is a Stitch-style learner, not a DreamCoder-style one. It has no neural recognition model; its library is grown by periodic anti-unification mining rather than a wake-sleep loop. The DreamCoder/Stitch line is the closest pre-neural library-learning prior art, and B5 is the clebran project's instantiation of that prior. The framing matters: B5 is the same library mechanism as the multi-agent arms, run by a single agent — so the comparison isolates the effect of centralisation from the effect of the library mechanism itself.*

arm	what	evo-solved /30
B5	**single pop + central library**	**2.65 ± 1.31**
B0	single population	2.25 ± 1.07
B7	recovered-primitive, unconstrained	1.85 ± 1.04
B6	recovered-primitive, constrained	1.80 ± 1.06

comparison	Δ (se)	verdict
B5 – B6	**+0.85 (0.38)**	**SIGNIFICANT (~2.2 se) — the central library is *better* than the recovered-primitive channel**
B5 – B0	+0.40 (0.38)	directional (~1.1 se), not significant
B5 – B7	+0.80 (0.39)	directional, ~2.0 se — would reach the bar with more seeds
B5 – B1	+1.20 (0.37)	would reach the bar with more seeds

The vocabulary metrics tell the same story from a different angle. B5's library is more compositional (topographic similarity 0.33 vs B6/B7 0.14–0.15) and more cross-task-reused (0.15 vs 0.05). The single agent's mined vocabulary is genuinely better than any multi-agent arm's.

[exp: 2026-07-22_b5_central_library]

Honest reading of the B5 result. B5 is the *best arm* on the 20-seed frontier; the single formally-tested B5 vs B6 contrast is significant at ~2.2 SE; and the qualitative vocabulary-quality metrics support the ranking. Two caveats are in order. First, the B5 – B0 contrast (+0.40, 1.1 SE) is *directional, not significant* — the paper's "central library adds to single-population search" claim is supported by the vocabulary metrics but not by the headline solve-

count. Second, B5's library pool is built from a single population with ~2.65 mean solves, while B6/B7's per-lineage libraries are built from ~0.6–0.7 mean solves per lineage — a ~4× difference in library-pool size that may itself explain the vocabulary-quality gap. The cleanest follow-up control is a *B0 + library* arm (a single population with the same compression miner, no multi-agent framing), which is the obvious next measurement (§7).

Why the central library wins, mechanistically. The full ranking tells a coherent story. Three forces explain it:

> *B5 (2.65) > B0 (2.25) > B7 (1.85) ≈ B6 (1.80) > B2 (1.60) > B1 (1.45)*
> *└ central library ┘ └ multi-agent + channel ┘ └ fragment ┘ └ isolated ┘*

1. **Splitting compute is expensive** (the *divergence tax*): 3 isolated lineages (B1 1.45) fall well short of one population (B0 2.25).
2. **Communication pays back only part of that tax** (B6/B7 ≈ 1.8), never reaching B0.
3. **The *same* library mechanism, run centrally** (B5), not only avoids the tax but *adds* to single-population search — landing on top.

The reason B5's library is better is visible in the metrics: a single agent sees *all* tasks, so its mined abstractions generalise across them; a lineage that sees only the subset it happened to solve mines task-idiomatic abstractions. Divergence — the premise of H1 — is precisely what *degrades* the shared vocabulary. Partial views yield task-idiomatic abstractions.

The project's central claim — that divergent lineages sharing over a constrained channel beat the alternatives at matched compute — is not supported; the strongest control wins. This is a *trustworthy* negative result, exactly the outcome the design was built to make believable.

5.6 Curriculum channel (B8/B9) — even an engaged channel does not beat B5

A fourth payload type — the *curriculum* — is the §7b hypothesis: knowledge crosses as *problems to solve*, not code, by exporting synthetic tasks generated from a lineage's own DSL and ordered by prerequisite structure. An order-aware receiver gates deeper tasks behind their prerequisites being solved. 20 seeds, 4M evals.

arm	what	evo-solved /30
B5	single pop + central library	**2.65 ± 1.31**
B0	single population	2.25 ± 1.07
B7	recovered-primitive, unconstrained	1.85 ± 1.04
B6	recovered-primitive, constrained	1.80 ± 1.06
B9	**curriculum, unconstrained**	1.65 ± 0.75
B2	fragment channel	1.60 ± 0.99
B8	**curriculum, constrained**	1.60 ± 0.75
B1	isolated	1.45 ± 0.94

comparison	Δ (se)	verdict
B8 – B1	+0.15 (0.27)	curriculum transfer barely beats isolation, **ns**
B8 – B6	−0.20 (0.29)	curriculum is *weaker* than the recovered-primitive payload, ns
B8 – B9	−0.05 (0.24)	**null** — the constraint doesn't matter for teaching sets either
B8 – B5	−1.05 (0.36)	curriculum is significantly *worse* than B5 (~2.9 SE)

The mechanism *did* work. Lineages received ~31 teaching tasks each and solved **70–78% of them** (B8 23.8/30.6 = 78%; B9 21.9/31.2 = 70%). The students demonstrably *learned the concepts their peers taught*, in the prerequisite order, exactly as designed. And it bought essentially nothing on the real tasks: +0.15 over isolation, below every other channel payload, and significantly below B5.

The bottleneck is not acquiring a taught concept — that worked. It is that *learning a concept from demonstrations does not transfer to new tasks that need it*. A lineage can solve the synthesised task showing `vmirror`, and still not deploy `vmirror` where a frontier task requires it. This is the L2T transfer-vs-reconstruction gap appearing at the level of *whole concepts* rather than single programs.

Teaching also costs budget: the curriculum arms solve ~24 auxiliary tasks each, compute diverted from the tasks that count. Under matched compute, teaching is close to a wash.

[exp: 2026-07-23_frontier_curriculum]

A caveat on one number: `reuse` reads 0.00 for B8/B9 because `reuse_rate` measures solutions containing a *received fragment or abstraction*; a curriculum payload transmits neither, so the metric is inapplicable. The curriculum-specific engagement metric is the aux_received / aux_solved table.

5.7 The teach→test probe and the amortization correction

The B5 result forced two follow-up questions. Was the *channel* that failed, or the *measurement*? And does the curriculum's *engagement* (70–78% of teaching tasks solved) translate to anything measurable on the real tasks?

5.7.1 The teach→test probe (without a cold baseline)

A controlled teach→test protocol separates *acquiring* a taught concept from *transferring* it to a novel task needing the concept. Phase 1: evolve on a curriculum only, then mine the resulting library. Phase 2: spend the rest of a shared total budget (500k) on the target only, carrying the phase-1 library forward.

- **Taught arm** — curriculum is the target's own prerequisite decomposition (an *oracle-relevant* curriculum: maximally well targeted, an upper bound on transfer).
- **Control arm** — an unrelated program's decomposition, truncated to the same item count, same seed, same total budget.
- **8 targets × 5 seeds → 34 paired (target, seed) runs per arm.**

	taught	control
target solve rate	38.2% (13/34)	32.4% (11/34)
curriculum learned	50%	46%
mean teach budget used	82.6k	90.2k
mean test budget available	417k	410k
evals-to-solve, fraction of available budget	**0.150**	**0.318**

Two findings of opposite sign. *Solve rate is null* (4 vs 2 discordant pairs, binomial p = 0.688). *Speed is real and significant* (15.0% vs 31.8%, permutation p = 0.025) — the taught arm is roughly 2× faster on the runs that solve. Both arms spent almost the same teaching budget, so the test budget was almost the same in both; the taught arm genuinely finds the target sooner.

A caveat on the conditional comparison. The "evals-to-solve" comparison is computed on the *runs that solved* in both arms (n = 9 all-solved pairs); the conditioning set is not symmetric, because the discordant pairs split 4-vs-2 in favour of the taught arm. The conditional comparison can therefore overstate or understate the ITT effect. The next section (§5.7.2) reports a *cold* baseline, which is the more reliable verdict.

[exp: 2026-07-23_transfer_probe]

Concept transfer is real — it accelerates search — but acceleration is not the same as capability. A relevant curriculum makes the target twice as cheap to find, and changes almost nothing about which targets are findable at all. That precisely explains the curriculum channel's null (B8 − B1 = +0.15, ns): the channel's benefit is *speed*, but the arm metric is *solve count at fixed*

budget, and the speed only converts to solve count for tasks sitting *near* the budget frontier, of which the frontier subset has very few.

5.7.2 The amortization correction (cold baseline)

The probe lacked a *no-teaching baseline*. Adding a *cold* arm (empty curriculum, full budget on the target) and merging the three (same targets, seeds, budget) gives the corrected three-way verdict. The means below are over the **n = 9 pairs solved by all three arms** (the only estimand on which all three are defined); the comparison is therefore "conditional on solving," not ITT. The conditional-vs-ITT distinction matters: the n = 34 ITT comparison is more variable, and the n = 9 conditional is what makes the variance tight enough to give the p = 0.97 verdict below.

arm	curriculum	target-solve evals (T)	n	solve rate (n=34)
cold	none	73,152 (median 61,866)	9	35.3%
taught	target's own (relevant)	72,166 (median 72,082)	9	38.2%
control	unrelated (irrelevant)	116,067 (median 83,221)	9	32.4%

paired difference	Δ (n=9)	perm p	reads as
taught – cold	**−986**	**0.967**	relevant teaching ≈ no teaching
control – cold	+42,915	0.269	irrelevant teaching is *slower* than no teaching
control – taught	+43,901	—	*this is what the probe measured*

The probe's 2× was the size of the harm to the mis-targeted control, mislabeled as a benefit. The honest verdict is the opposite of the probe's headline: *relevant teaching is roughly neutral; irrelevant teaching causes negative transfer*.

[exp: 2026-07-23_amortization]

The amortization story is also cleanly closed. Per student, taught total is L + T = 128,942 vs cold U = 73,152 — the per-student condition L + T < U is violated by ~56k. Amortization spreads the *teacher's* one-time cost C across N students; if each student is individually worse off, no N rescues it. The crossover formula `N > C / (U − L − T)` has no solution. Sharing a curriculum among many agents *cannot* pay off here, because the thing being shared does not make an individual agent cheaper.

Two caveats. The protocol wastes teaching budget (mean L = 56,775 vs median 7,561 — a few runs burn the full 150k cap without finishing). An early-stopped teach phase would lower L, but since T ≈ cold already, it cannot turn the per-student condition positive. The taught curriculum is the target's own solver decomposition — the *oracle upper bound*. Real peer curricula are strictly worse targeted, so ≈ 2× is a ceiling on the speed benefit, not a typical value.

5.7.3 The order experiment — the first significant process effect

The teach→test and amortization results close the *content* question: relevant teaching is neutral, irrelevant teaching harms. They say nothing about *process*. The order experiment is the pristine test of the "how you learned it" thesis: it holds the curriculum *content* fixed — a target's k prerequisite sub-tasks — and varies *only* the order in which they are acquired. Four arms, identical content and identical total budget (500k), with cost-to-solve as the primary endpoint: **cold** (no curriculum), **forward** (foundations first, gated), **reverse** (hardest first, gated), **shuffled** (all steps at once, ungated). The three teaching arms differ only in the prerequisite `score` handed to the order-aware gate — so order is isolated and nothing else changes. 9 targets (depth ≥ 3, ≥ 2 grid prerequisite steps) × 6 seeds → 216 runs.

order	curriculum acquired	target solve rate
forward (foundations first)	**47%**	**48.1%**
shuffled (unordered)	45%	38.9%
reverse (hardest first)	**23%**	33.3%
cold (no curriculum)	—	29.6%

Paired McNemar (by target × seed), the correct test for the paired binary solve outcome:

comparison	wins	p	reads as
forward – reverse	9 vs 1	**0.021**	right order ≫ wrong order (same content)
forward – cold	13 vs 3	**0.021**	well-ordered teaching beats none
forward – shuffled	7 vs 2	0.180	ordering vs merely-available: **ns**
reverse – cold	5 vs 3	0.727	**wrong-order teaching ≈ no teaching**

Two significant results, and both are about the *wrong* order. The mechanism is mechanistically legible: teaching foundations-*last* lets the learner acquire only **23%** of the same curriculum — half of forward's 47% — because a gate that demands the hard composition before its parts stalls, and you cannot learn a composition before the pieces it is built from. That halved acquisition shows up as a significantly lower solve rate: forward beats reverse ($p = 0.021$), and reverse is *no better than not teaching at all* ($p = 0.727$). Same content, same budget, reversed order → the curriculum's value is destroyed.

But *fine-grained ordering is not the active ingredient*. forward vs shuffled is not significant ($p = 0.18$), and shuffled acquires 45% ≈ forward's 47% — because when all steps are available at once, the fundamentals are the *easy* ones and get solved first anyway. The learner self-orders; strict prerequisite gating only matters when it can do *harm* (reverse), not when it enforces an order the learner would have found regardless.

As with teaching-vs-not (§5.7.2), the effect is on *capability*, not speed: every paired evals-to-solve contrast between the order arms is null (forward – shuffled = +16k, p = 0.41). Order changes *which* targets are reachable — through how much of the curriculum is acquired — not how fast an already-reachable target is found. Different interventions surface on different endpoints: teaching- vs-not lived in speed, order lives in capability.

[exp: 2026-07-24_order_process]

Scope and one honest tension. forward – cold is significant here (p = 0.021) where the amortization taught – cold was neutral (§5.7.2). The difference is target selection: this experiment filters to targets with ≥ 2-step decompositions, where order can matter; amortization did not. So "well-ordered teaching helps" is scoped to targets with real compositional structure, not tasks in general — a qualification, not a contradiction. The discordant counts are small (9–17 pairs); the two significant results are the paired forward-vs-reverse and forward-vs-cold, both p = 0.021. A preliminary partial-data read had shown a larger forward-vs- shuffled gap that regressed to non-significance at full n — a caution recorded against reading early cells.

This is the project's **first significant process effect** and its first empirical purchase on the reorientation (§6.1). It is a *qualified* yes for "how you learned it": on compositionally- structured tasks, the order of acquisition significantly changes capability with content held constant — asymmetrically, *don't teach backwards* matters more than *teach in exactly the right order*.

5.8 Vocabulary quality

A summary of the vocabulary metrics across the multi-agent arms and the central-library control. Vocabulary metrics are only defined for arms that grow a library (i.e. not for B0, B1, or B2). The metrics supplement the solve-count headline with "how good is the abstract structure?" probes.

arm	vocab size	used-rate	cross-task	received-used	topsim
B6 (recovered, constrained)	4.5	0.27	0.05	1.12	0.14
B7 (recovered, unconstrained)	5.6	0.26	0.05	1.42	0.15
B5 (central library)	—	—	**0.15**	—	**0.33**

Three things to read off the table. *received-used is > 1* for B6 and B7 — lineages use *more than one* received primitive per solve, on average, a real signal that the channel is engaged. *cross-task reuse* is three times higher for B5 (0.15) than for B6/B7 (0.05): a single agent's mined abstractions generalise across more tasks. *topographic similarity* is more than double for B5 (0.33 vs 0.14–0.15): the central-library vocabulary is more compositional. The two metrics agree: the single-agent library is *better*, not just *more abundant*.

> *A note on the two topsim numbers in the paper: §3.6 reports topsim 0.53 for the miner run over all 400 Hodel reference solvers; the 0.33 reported here is B5's actual run on 20 seeds × ~1.3 mean solves. The 0.53 is a miner-quality number; the 0.33 is a run-quality number.*

The "divergence degrades vocabulary quality" finding is the mechanistic explanation of the B5 win: a single agent sees every task, so its mined vocabulary generalises across them; a lineage that sees only the subset it happened to solve mines task-idiomatic abstractions. The premise of H1 (divergence is an asset) is supported — lineages *do* diverge — but divergence in this substrate is a *liability* for vocabulary quality, not an asset.

6. Discussion

The headline is sharp. The discussion asks what the headline *means* — for the clebran design, for the clebran hypothesis, and for the broader question the experiment was supposed to be about. We organise the discussion around four claims: the *reorientation* (§6.1), the *divergence tax* (§6.2), the role of *measurement* (§6.3), and the *scope* of the result (§6.4). A short note on what the result implies for related literatures (§6.5) closes.

6.1 The reorientation: not what you know, how you learned it

The clebran hypothesis was that communication, in the form of a bandwidth-limited channel between divergent populations, would improve the *skill* of a system at matched compute. The data argue that the better purchase is *concentration* — one agent that sees every task and reuses its own mined abstractions strictly dominates several agents that see task slices and share. Communication has a small, real, hard-to-detect effect that lives in evals-to-solve rather than in solve count, and is more than wiped out by the divergence tax.

The reorientation of the question is forced by two facts. The first is the *transfer probe's corrected verdict* (§5.7.2): relevant concept teaching is neutral on cost, irrelevant teaching is harmful, and the per-student condition L + T < U fails by ~56k. The thing that was supposed to be the central mechanism of clebran (a curriculum that *teaches* what the sender learned) does not, in this substrate, at matched compute, make an individual agent cheaper to reach a target capability.

The second is the *endpoint*. The transfer probe's apparent 2× speedup was real but lived in evals-to-solve, not in solve count. Re-scoring on cost exposes that curriculum teaching's benefit is invisible under solve count at fixed budget, and only converts to solve count for tasks *near* the budget frontier — of which the frontier subset has very few. Almost every prior null in the project is therefore a null on a *solvable-frontier* metric, not a null on *acquisition efficiency*. The instrument for cost-to-solve was already in the logs; it was not the metric.

Chollet's *On the Measure of Intelligence* warns that *unlimited priors or unlimited training data let you "buy" skill*, masking a system's actual generalisation power. The clebran design was deliberately small, fully instrumented, and pre-neural so that acquisition cost is metered, not purchased. The honest reading of the headline is that even on this small substrate, the multi-agent-with-channel framing is dominated by the central- library framing. The next question is therefore the one *On the Measure of Intelligence* is really about: how *acquisition-efficient* is the system, and what design choices move that number?

The reorientation is not only a proposal: the order experiment (§5.7.3) is a first empirical purchase on it, and it comes back *positive*. Holding the taught *content* fixed and varying only the *order* of acquisition produces a significant capability difference — teaching a curriculum foundations-*last* (reverse) halves how much of it is acquired (23% vs 47%) and is no better than not teaching at all, while foundations-first (forward) is significantly better (paired $p = 0.021$). This is the cleanest statement the substrate can make of "how you learned it": the *same* knowledge, acquired in a dependency-violating order, is worth significantly less. The effect is asymmetric — *don't teach backwards* matters, but fine sequencing beyond "make the fundamentals available" does not (forward ≈ shuffled, ns), because an adaptive learner self-orders when all steps are on offer. So the process axis is real but, in this substrate, shows up mainly as a *failure mode to avoid* rather than a lever to optimise. That it appears at all — as the project's only significant *process* effect, against a wall of null *content* effects — is what marks the axis as the one worth pursuing.

A capability audit (§6.4) frames the rest of the answer. The process-level capabilities that an "acquisition-efficient" learner would need — self-directed practice, active choice of what to practise, retention without rigidity, reflection on *how* something was learned — are the gaps. None of them is fully present in the clebran design; together they describe the substrate the reorientation would actually need. The order result is a proof of concept that the *axis* is measurable here; the capability audit is why measuring it *richly* needs a more capable learner than pure GP, for which order this cheaply self-corrects.

6.2 The divergence tax

The single clearest mechanism in the data is the *divergence tax*: splitting a fixed compute budget across N populations yields fewer solves than spending the whole budget on one population. B1 (3 isolated lineages, 1.45) is *worse* than B0 (single pop, 2.25). The mechanism is straightforward: each lineage sees a third of the population generations, and the small populations are more vulnerable to premature convergence on local optima.

Communication pays back part of the tax. B6/B7 (≈ 1.8) are better than B1 (1.45), but never reach B0. The *received-primitive* signal is real (received-used ≈ 1.1–1.4, NEO-style), and the recovered-primitive channel does help isolated lineages solve more — but the gain is small (0.35 mean solves over B1 at 20 seeds) and the gap to B0 is unchanged.

A central library *avoids* the tax and is *consistent with* adding to single-population search. B5 (2.65) > B0 (2.25) is directional (+0.40, 1.1 SE) — not formally significant — but the qualitative claim survives: B5's library is more compositional (topsim 0.33 vs 0.14–0.15) and more cross-task- reused (0.15 vs 0.05). The reason is in the metrics: a single agent sees every task, so its mined abstractions generalise across tasks; a lineage that sees only the subset it solved mines task-idiomatic abstractions. Divergence is precisely what *degrades* the shared vocabulary — partial views yield task-idiomatic abstractions, even when the partial views are communicated and combined.

Caveat on the B5 − B0 attribution. B5's win over B0 is *not* isolated to the library mechanism: B5 also has a ~4× larger library pool (built from a single population with ~2.65 mean solves, vs B6/B7's ~0.6–0.7 per lineage) and is run on a single agent rather than three. The cleanest experimental isolation of the library mechanism is a B0 + library arm (single population, same compression miner, no multi-agent framing), which is the obvious follow-up (§7). Until that control is run, the paper's claim is that *centralisation plus the library mechanism* dominates *divergence plus any channel*, not that the library mechanism alone is what wins.

The negative result has a positive corollary: the design of the *library* (the compression miner, the MDL pressure, the Stitch-style library growing mechanism) is the part of clebran worth keeping. The multi-agent framing, the channel, the constraint regime, and the curriculum payload are the parts worth re-thinking. B5 is the strongest "abstraction reuse without divergence" result, and it is consistent with the broader library-learning literature: a Stitch-style learner on a hard synthesis problem is a strong baseline, and any multi-agent framing has to *beat* it, not just match it.

6.3 The role of measurement

The clebran design's most important asset is its measurement rig, and the headline is the rig's most important output.

The match-compute protocol audits each arm's recorded candidate-evaluations to the per-arm budget and reports the spread. Audited spreads are ≤ 0.27% across all reported multi-arm runs; for the headline 20-seed frontier runs the spread is 0.10%. The per-task RNG substream scheme (`Random(f"{seed}:{task_id}")`) makes a single-task probe predict full-run behaviour exactly, so a unit of debugging work on a single task is a unit of debugging work on the run. The deterministic interpreter and the `Program.from_json` round-trip let every experiment be re-run from a frozen commit and reproduce the same summaries.

The rig is what makes the null trustworthy. A cheap critique of a null result in evolutionary program synthesis is "you just didn't give the channel enough of a chance" — more seeds, more budget, more bandwidth. The power check (§5.4) doubled the seeds (8 → 20) and raised the budget (3M → 4M); the effect *shrank* (B6 − B2 fell +0.50 → +0.20) and the standard error

narrowed (0.48 → 0.32), so the result is real-but-small rather than a near-miss on significance. The dose-response (§5.3) tested the constraint versus unconstrained range from `none` to `uncon` and found the channel's *crowding mechanism is absent* because AOS self-constrains a failing channel.

Two methodological lessons are worth stating explicitly. First, *report cost-to-solve alongside solve count*. The transfer probe is the dramatic example: a 2× speedup in evals-to-solve is invisible under solve count at fixed budget if the budget frontier has no tasks near it. Adding the cost endpoint is a cheap fix; it does not undo the headline ranking (B5 still wins), but it changes the discussion from "no effect" to "an effect that is invisible to the standard endpoint."

Second, *the cold baseline is non-negotiable*. The transfer probe's headline was wrong because it compared taught to a control arm that was *mis-targeted* and therefore *harmed by the channel*. Adding the cold arm revealed the confound and corrected the verdict. The rule generalises: in any "intervention helps" experiment, a *do-nothing* arm is the only way to distinguish a real benefit from a *harm to a mis-targeted control* masquerading as a benefit. The clebran project's later arms (B5, B8/B9) explicitly include a cold reference for exactly this reason.

What it took to make the rig reliable. The spec's Stage-1 gate — B0 solves all 10 near-atomic tasks on *every* seed — forced a series of mechanisms, each fixing a diagnosed failure mode. The full debugging trail is itself a record of what pure-reward program evolution needs to be reliable. In the order the fixes were discovered:

1. A union-type-flow bug that made `vmirror(I)` compositions unreachable: fixed by the union flow-through rule (§3.1).
2. A reward exploit where blank grids out-scored content: fixed by the foreground/background-balanced reward (§3.4).
3. Restart amnesia: fixed by the hall-of-fame restarts (§3.3).
4. A shared-RNG context sensitivity: fixed by the per-task RNG substreams (§3.3).
5. A round-robin scheduling starvation on near-miss tasks: fixed by the progress-weighted scheduler (§3.3).
6. A phase-1 fragment splice that poisoned 15% of offspring on needle tasks: fixed by the AOS self-suppression of fragment splice where library material is irrelevant (§3.3).
7. A literal-tuple → list `Program.from_json` bug introduced by the abstraction replay support: fixed by an explicit tuple coerce.

Each fix is identical across arms, so the H2/H3 comparison is not biased by which fix landed first. The Stage-1 debugging trail is the most direct evidence the rig is reliable: a non-trivial set of mechanisms had to be added to make a stochastic GP engine pass a deterministic gate, and every mechanism is now part of every arm.

6.4 Scope: what the result is for, and what it is never for

The scope of the result is narrower than its surface. We state both halves explicitly.

Good for

- *Amortising known competence* — making the 2nd…Nth agent acquire a capability ≈ 2× cheaper, when the curriculum is oracle-relevant and the budget frontier has marginal tasks. The clebran data do not show this on the current frontier, but the teach→test probe shows the *effect exists*; what it lacks is the right distribution and the right setup.
- *Distributing competence cheaply and substrate-agnostically* — curricula are compact and, unlike code, assume no shared representation. The clebran *curriculum* channel is the payload type most likely to generalise across learner architectures.
- *Bootstrapping a starting vocabulary* — so that new agents do not begin at zero. A small, recovered abstraction library installed at the start of a run is a clean experimental manipulation, and the clebran data suggest it would be cheaper than a curriculum.
- *Interpretability / audit* — a curriculum is a human-readable statement of what an agent knows. The clebran solver card already renders every evolved program's step-by-step execution trace; a curriculum extends that audit to the *inter*-agent layer.

Never for

- *Discovering abstractions nobody has found* — neither clebran mechanism, taken alone, can create information not present in the teacher or in the population. The library-mining component is the *only* mechanism that can discover abstractions; the channel and the curriculum are *transport* mechanisms. The headline falsification is precisely that, in this substrate, transport does not help even when discovery is shared.
- *Raising the ceiling on what is solvable at all* — the teach→test probe shows that a relevant curriculum makes a target cheaper, not more reachable. Speed, not capability.
- *Beating concentrated compute at discovery* — the B5 control is the single best arm. Splitting compute is a tax, and communication does not pay it back fully.
- *Tasks outside the curriculum's vocabulary coverage* — the amortization result shows that mis-targeted teaching causes *negative transfer* (irrelevant teaching is slower than no teaching). This is the most important scoping result available: a taught vocabulary biases later search, so an agent that internalised vocabulary A may search *worse* than a naive agent on a task needing disjoint vocabulary B.

One-line scope. *This is a compression-and-transport layer for competence, not an engine of novelty*. A system that must *discover* should look like B5 — one learner, maximum compute, maximum task exposure. Teaching is what you bolt on afterwards to make that expensive competence cheap to replicate.

A *capability audit* for what a process-aware learner would need before "how you learned it" can be studied cleanly. The audit is a "what would have to be true for the reorientation to be

operational?" list; the clebran substrate is missing several of the rows.

capability	status in clebran
core-knowledge priors (objectness, geometry, number, agentness)	**have** — encoded in Hodel's DSL
abstraction / compression (MDL, library learning)	**have** — compression miner, MDL pressure
communication & teaching	**have** — 4 payloads, prerequisite-ordered curricula
honest measurement (matched compute, baselines, seeds)	**have** — the repo's real asset
acquisition efficiency as the objective	instrument exists, **not the metric**
self-directed practice / dreaming	**missing**
active learning — *choosing* what to practise next	**missing**
continual learning without rigidity (plasticity)	**missing, and possibly already violated**
compositional generalisation to novel combinations	**missing** (no OOD split)
generalisation to genuinely unseen tasks	**missing** (eval split untouched)
meta-learning — adapting the *learning algorithm* itself	**missing**
reflection — credit assignment over *how* it learned	**missing**, and the least-explored anywhere

The last row is the sharpest form of the thesis and is largely open in the literature: systems that model *their own acquisition process* and improve it. Everything above it is prerequisite scaffolding.

6.5 What this means for related literatures

Three implications for the literatures the clebran design draws on.

For *emergent communication*, the result is a clean refutation of the naive information-bottleneck story in a typed symbolic setting. Bandwidth-limited channels between divergent agents do not, in this substrate, induce better compositionality at matched compute. The reason is *adaptive operator selection*: an adaptive learner self-constrains a failing channel, so the information-bottleneck argument does not bite. The composi- tionality question is *still open* for non-adaptive learners and for the right evaluation setup; the clebran data close only the *adaptive* variant. The result is in direct conversation with the cultural-evolution theory of language (Kirby, Tamariz, Smith & Cornish, 2015; Brighton & Kirby, 2006), which argues that compression pressure *does* drive compositional structure in human languages. The clebran H3 null is a clean empirical counter-example in a typed-symbolic substrate: under adaptive

selection, the constraint and the unconstrained channel produce indistinguishable vocabularies. The H3 null does not refute the cultural- evolution account (which involves iterated cultural transmission rather than a single channel between populations) but it does argue that the simple information-bottleneck story is not the whole story even in a non-neural, deterministic setting. The closely-related result by Bouchacourt & Baroni (2018) — that successful emergent communication does not entail compositionality in neural referential games — provides a non-clebran counter-example to the same simple story, and the clebran H3 null is a typed-symbolic re-instantiation of the same gap.

For *library learning*, the result is consistent with the central library mechanism being a strong baseline. B5 is the single best arm and beats every multi-agent arm; the vocabulary metrics support the ranking; the only formally- tested contrast (B5 − B6 = +0.85, ~2.2 SE) is significant. The clebran design's compression miner and MDL pressure are not novel in themselves (both inherit from Stitch and DreamCoder), but the match-compute comparison against divergent multi-agent baselines is, and the win is significant.

For *L2T/NEO*, the result is *consistent with* the transfer metric framing. NEO's headline test (transfer, not reconstruction) reappears in the teach→test probe: students *learn* the taught concept (reconstruction) but do not deploy it on novel tasks (no transfer). The clebran data add a sharp *empirical* example to the L2T claim that reconstruction is not the right endpoint. They also add a sharp *negative* example: the *received-used* signal in the recovered-primitive channel is real (lineages use received primitives, on average, more than once per solve) but the *transfer* signal is not — those used primitives do not improve solve count at matched compute in the multi-agent setting, and a single agent that mines its own primitives (B5) builds a strictly better library.

The combined picture is consistent and blunt. In this substrate, on this frontier, at matched compute, *communicating learned content — as code or as curricula — does not help, and mis-targeted communication hurts*. The remaining live thread is not *content* transfer but *process* (the reorientation): whether the *way* a system learns can be improved and measured on the acquisition-efficiency axis, where — unlike in the multi-agent comparison — the instrument shows real structure.

7. Limitations

The clebran experiment is small by design, and the smallness is both a strength and a limitation. We list the limitations in order of how much they bound the result's claim.

Substrate. The experiment is scoped to ARC-AGI-1 program synthesis over Hodel's 160 typed primitives, with a pure evolutionary search and no neural component. The negative result — that divergent multi-agent populations with a constrained channel are dominated by a single agent with a central library — is established on this substrate only. The transfer of the result to other

program-synthesis settings (e.g. string transformation benchmarks), to other multi-agent settings (e.g. emergent communication referential games with non-adaptive learners), and to language-model agents is an empirical question for each. The result is most relevant where the substrate is a small, fully-instrumented one and the learner is adaptive.

Frontier. The frontier subset is 25 D2 + 5 D3 tasks (depth 2-dominated, because D3 is beyond reach at 4M evals). The solvable frontier is therefore narrow, and the headline effects are 1–3 evolution solves/seed. Most comparisons are underpowered by construction. The single significant comparison is B6 − B5 = −0.85 (~2.2 se); the rest of the ranking is *consistent* across runs but not per-comparison significant.

Solve count vs cost-to-solve. The headline endpoint is *evolution-phase solves* (count) at a fixed candidate- evaluation budget. Cost-to-solve is a *secondary* endpoint that has been re-scored for the teach→test probe and the amortization result; the multi-agent headline ranking has not been re-scored on cost. A cost-axis re-scoring of the B0–B9 frontier ranking is a small, cheap follow-up that should be done before publication; it may not change the ranking, but it would change the discussion of how the ranking was *measured*.

Single round of sharing. The experiment runs *one* round of communication. The iterated form ($\text{curriculum}_1 \rightarrow \text{agents}_2 \rightarrow \text{curriculum}_2 \rightarrow \ldots$) — the actual open-endedness question — is unaddressed. The single round is what the *channel* baseline in the emergent-communication literature typically uses, and it is the right unit for a falsification experiment; it is not the right unit for a *positive* result on competence compounding.

No compositional OOD split. The L2T compositional-OOD and length-OOD splits (Baek et al., 2026) — controlled distributions over the compositional depth and length of the target program — are an interesting generalisation probe. The clebran experiment does not run them; the headroom for positive transfer effects is therefore unknown.

No eval-split generalisation. All reported results are on the training split. The 400-task eval split is reserved for the "does this generalise to genuinely unseen tasks" question and has not been touched. The clebran design's library- mining mechanism is the part of the project most likely to show *or fail to show* generalisation; the eval split is the obvious next test.

Noisy underpowered comparisons. Several headline comparisons (B6 − B1, B6 − B2, B8 − B6) are at 1.0–1.4 standard errors — directionally consistent across runs but not per-comparison significant. The cleanest interpretation is the *ordering* of the arms, not any single pairwise contrast.

Compression miner is not optimal. The compression miner is Stitch-style anti-unification, which is fast but sub-optimal compared to a full search-based compression algorithm. A more compressive miner might produce a better library for B5 (raising the bar further for any multi-agent arm). The clebran result therefore *understates* the advantage of the central library; the falsification is robust to this concern.

Missing B0+library control. The B5 win is confounded between three factors: (a) centralisation (one agent vs three lineages), (b) library-pool size (B5's pool is built from ~2.65 mean solves, ~4× the per-lineage pool of B6/B7), and (c) the library mechanism itself. The cleanest isolation of the library mechanism is a *B0 + library* arm (a single population with the same compression miner, no multi-agent framing), which is the obvious follow-up that would isolate the contribution of the library mechanism from the contribution of centralisation. This arm is listed in the project's roadmap but has not been run; the B5 headline should be read as "centralisation plus the library mechanism dominates divergence plus any channel," not as "the library mechanism alone is what wins." Until the B0+library control is run, the clebran paper's library-learning framing (§6.2, §6.5) is supported by the vocabulary-quality metrics but not by a clean solve-count contrast.

No statistical power analysis. The standard errors are *descriptive*, not a power analysis. A power analysis would let us say "X more seeds are needed to detect effect size d at $\alpha = 0.05$, $\beta = 0.2$" — useful for sizing follow-up experiments, not done here. The current seed counts (5 on Stage-1, 20 on the frontier) are *large enough* to detect the B6 − B5 effect, but not large enough to detect smaller channel-driven effects even if they exist.

Cross-task reuse metric is conservative. Cross-task reuse is the fraction of pairs (task, abstraction) where the abstraction is used on a task *other than* the one it was mined from. The metric does not count an abstraction that is *available* in the lineage's library but unused on a later task; the right metric for "is the library useful" may also include the *availability* rate. The reported numbers are therefore a *lower bound* on cross-task utility.

The order experiment is small and narrowly scoped. The completed run is 216 rows — 9 targets (depth ≥ 3, ≥ 2 prerequisite steps) × 6 seeds × four order arms (forward / reverse / shuffled / cold, §5.7.3). Two paired comparisons are significant (forward − reverse and forward − cold, both McNemar $p = 0.021$), but the discordant counts are small (9–17 pairs), the significant effects are on *capability* (solve rate) rather than the pre-registered cost-to-solve endpoint (all order-vs-order evals contrasts are null), and the significant "well-ordered teaching helps" result (forward − cold) is scoped to targets with ≥ 2-step decompositions — it does not generalise to tasks without compositional structure, where amortization found teaching neutral. A preliminary partial-data read had shown a larger forward-vs-shuffled gap that regressed to non-significance at full n, a caution against reading early cells. The result is reported as the first *process* effect, honestly bounded, not as a general claim that curriculum ordering optimises learning.

Honest reporting of nulls. The project has a track record of correcting its own nulls: the transfer probe showed a 2× speedup, the amortization experiment showed the speedup was an artifact of a missing cold baseline, and the FINDINGS and README were updated. The clebran design's *measurement rig* (matched compute, audited budgets, strong baselines, paired designs, permutation tests) is the part of the project that makes the nulls trustworthy; it is also the part that future work should reuse.

8. Conclusion

We set out to test whether divergent populations of DSL programs on ARC-AGI-1, exchanging successful subroutines through a constrained channel, would beat the alternatives at matched compute. The experiment delivered a falsification. On a 30-task frontier subset at 4M candidate-evaluations per run, with audited compute spread of 0.10%, the single significant comparison in the entire study is **B6 − B5 = −0.85 (~2.2 se)**: a single population that grows a central abstraction library from its own solutions (a DreamCoder/Stitch-style learner) is significantly better than the recovered-primitive channel between divergent populations. Every multi-agent arm lies between isolation (worst) and the central library (best). The constraint-versus-unconstrained comparison is null across all four payload types, because adaptive operator selection self-regulates a failing channel. H1 is supported — lineages do diverge — but divergence in this substrate is a *liability* for vocabulary quality, not the asset the hypothesis assumed.

The result is meaningful only because the baselines are strong. The match-compute protocol audits each arm's recorded candidate- evaluations to the per-arm budget; the per-task RNG substream scheme makes single-task probes predict full-run behaviour; the deterministic interpreter and the 399/400 reference-solver reproduction gate make every evolved program auditable. The power check (8 → 20 seeds, 3M → 4M evals) confirmed the effect is small rather than a near-miss on significance.

A controlled teach→test probe then forced a reorientation of the question. A relevant curriculum makes a target ≈ 2× cheaper to find, but does not expand what is solvable; an irrelevant curriculum causes negative transfer. With a *cold* baseline (no curriculum) added, the honest verdict is the opposite of the probe's headline: relevant teaching is roughly neutral, irrelevant teaching hurts, and the per-student condition L + T < U fails by ~56k — a number no amortisation across N agents can rescue. The teaching's benefit lives in *evals-to-solve*, not in solve count, and is invisible under the standard endpoint at a budget frontier with few marginal tasks.

The reorientation is therefore from "does communication raise skill?" to "does the *way* a system learns change how well it can use what it learned?" That is the question Chollet's *On the Measure of Intelligence* argues is the right one, and the order experiment (§5.7.3) is the paper's first evidence that the substrate can answer it: with taught *content* held fixed, varying only the *order* of acquisition produces a significant capability difference (foundations-first beats foundations-last, paired $p = 0.021$), because a dependency-violating order halves how much of the same curriculum is acquired. The effect is asymmetric — *don't teach backwards* matters more than *teach in exactly the right order* — but it is real and significant, and it is the only significant *process* effect against a wall of null *content* effects. That contrast is the reorientation's warrant.

The substrate has *measurement* as its asset, and studying process *richly* requires capabilities it does not yet have: self-directed practice, active choice of what to practise, retention without rigidity, compositional generalisation, generalisation to genuinely unseen tasks, meta-learning, and reflection on *how* something was learned. The capability audit is a "what would have to be true?" list; the substrate is currently missing most of the rows — which is also why order self-corrects so cheaply here (a richer learner would not).

Three open work items are the most directly productive.

1. **Push the process axis past order.** The order result shows the axis is measurable but that pure GP self-orders too easily for fine sequencing to matter; the natural next tests are *active* choice of what to practise (a learning-progress teacher) and *retention without rigidity* — the plasticity question the amortization negative-transfer result suggests is already violated (a taught vocabulary may bias later search).
2. **9a: make cost the primary endpoint everywhere.** The teach→ test and order experiments already use cost-to-solve; the remaining pass is to re-score the *content* arms (B0–B9) on evals-to-solve + evals-per-solve. The data is already in the JSONL; it is an analysis pass, not new compute. Expect some prior nulls to wake up on the cost axis. Add the column to `metrics/report.py` and the frontier analyser.
3. **9b: point `clebran.curriculum` inward (dreaming).** The task generator already exists and is only ever pointed outward. After solving, a lineage could generate variants of its own solutions, train on them, and measure (a) vocabulary quality (topsim, cross-task reuse) and (b) whether later solves get cheaper. This is the single most implementable big missing capability.

The negative result is the paper's contribution. It is what the clebran design's baselines were built to make believable, and it is the result that any future work in the area has to address. The "how you learned it" reorientation is the project's open thread: the metric exists, the experiment is writable, and the question is sharper than the one we set out to ask. The clebran substrate — its measurement rig, its compression miner, its linear IR, its deterministic interpreter, its track record of reporting nulls — is the asset that lets the reorientation be made operational.

References

> *Numbered in citation order. Inline citations in the paper use the form `[author, year]`; this file is the canonical bibliography.*
>
> ***Important:** the first draft of this bibliography contained three hallucinated citations (refs 7, 8, and 25 in the original numbering) — the ArcMemo author list and title, the Kiela 2019 "compositionality challenge" reference, and the L2T/NEO paper title and first*

> *names. These were caught by the related-work reviewer (`reviews/03-related-work.md`) and have been replaced in this pass with the reviewer's supplied corrections.* ***The user should re-verify every replaced or newly-added citation against the source (arXiv / DOI / official page) before submission.*** *The corrections are flagged with `*(reviewer- supplied — verify)*` in the entries.*

1. **Chollet, F.** (2019). *On the Measure of Intelligence*. arXiv:1911.01547. The framing of intelligence as skill-acquisition efficiency over a scope of tasks.
2. **Chollet, F.** (2019). The Abstraction and Reasoning Corpus (ARC). https://github.com/fchollet/ARC-AGI.
3. **Hodel, M.** (2024). *arc-dsl: a typed grid DSL for ARC*. https://github.com/michaelhodel/arc-dsl. The 160 typed grid primitives and the 400 hand-written reference solvers that are the clebran substrate's backbone.
4. **Ellis, K., Wong, C., Nye, M., Sable-Meyer, M., Cary, L., Morales, L., Hewitt, L., Solar-Lezama, A., & Tenenbaum, J.B.** (2020). *DreamCoder: Growing generalizable, interpretable knowledge with wake-sleep Bayesian program learning*. arXiv:2006.08381.
5. **Bowers, M., Olaechea, T., constant, A., Drewniak, M., & Ellis, K.** (2023). *Stitch: A compositional framework for library learning*. arXiv:2211.16605.
6. **Lazaridou, A., & Baroni, M.** (2020). *Emergent Multi-Agent Communication in the Deep Learning Era*. arXiv:2006.02419.
7. **Brighton, H., & Kirby, S.** (2006). *Understanding Linguistic Evolution by Visualizing the Emergence of Topographic Mappings*. The original topographic-similarity paper; the metric the clebran project uses to measure compositionality of recovered vocabularies. *(reviewer-supplied — verify)*
8. **Lazaridou, A., Hermann, K.M., Tuyls, K., & Baroni, M.** (2018). *Emergence of Linguistic Communication from Referential Games with Symbolic and Pixel Input*. ICLR 2018. arXiv:1804.08784. Operationalises the topographic-similarity metric for emergent communication. *(reviewer-supplied — verify)*
9. **Chaabouni, R., Kharitonov, E., Bouchacourt, D., Dupoux, E., & Baroni, M.** (2020). *Compositionality and Generalization In Emergent Languages*. ACL 2020. arXiv:2004.09124. The cleanest empirical demonstration that compositional structure (as measured by various proxies) is decoupled from generalisation. *(reviewer-supplied — verify)*
10. **Bouchacourt, D., & Baroni, M.** (2018). *How agents see things: On visual representations in an emergent language game*. EMNLP 2018. arXiv:1808.10696. The canonical demonstration that successful emergent communication does not entail

compositionality — the original reference for the gap the clebran H3 null speaks to. *(reviewer-supplied — verify)*

11. **Li, F., & Bowling, M.** (2019). *Ease-of-Teaching and Language Structure from Emergent Communication*. NeurIPS

2019. arXiv:1906.02403. The topsim–generalisation decoupling: topsim rises in the "reset" regime, but topsim does not perfectly predict task transfer. *(reviewer-supplied — verify)*

12. **Kirby, S., Tamariz, M., Smith, K., & Cornish, H.** (2015). *Compression and communication in the cultural evolution of linguistic structure*. Cognition 141, 87–102. The information-bottleneck theory of language — the canonical "constraint drives compositionality" claim the clebran H3 null is in conversation with. *(reviewer-supplied — verify)*

13. **Baek, D., Lee, G., Baek, J., Lee, H., & Ahn, S.** (2026). *Learning to Theorize the World from Observation*. ICML 2026 oral. arXiv:2605.03413. *(Title, first names, and venue corrected from the first-draft hallucination; reviewer- supplied — verify against the arXiv listing.)* L2T/NEO is the primary theoretical anchor for the clebran transfer-vs-reconstruction decomposition.

14. **Kharitonov, E., Chaabouni, R., Bouchacourt, D., & Baroni, M.** (2019). *EGG: a toolkit for research in Emergence of lanGuage in Games*. arXiv:1907.00852. The EGG framework paper; clebran adopts the topographic-similarity and discrete-channel framing from EGG. *(reviewer-supplied — verify)*

15. **Poli, R., Langdon, W.B., & McPhee, N.F.** (2008). *A Field Guide to Genetic Programming*. The standard reference for the population-based GP literature the clebran lineage model inherits.

16. **Potter, M.A., & De Jong, K.A.** (1994). *A cooperative coevolutionary approach to function optimization*. The parallel-evaluations framing for multi-population GP.

17. **Skolicki, Z., & De Jong, K.A.** (2005). *The influence of migration sizes and intervals on island models*. The migration-policy literature that the clebran adaptive operator selection answers.

18. **Jaderberg, M., Dalibard, V., Osindero, S., Czarnecki, W.M., Donahue, J., Razavi, A., Vinyals, O., Green, T., Dunning, I., Simonyan, K., Fernando, C., & Kavukcuoglu, K.** (2017). *Population Based Training of Neural Networks*. arXiv:1711.09846. The modern neural-network-native analogue of island models. *(reviewer-supplied — verify)*

19. **Cully, A.** (2019). *Multi-Objective Quality-Diversity Optimisation*. In *Recent Developments in Evolutionary Optimisation in Modelling and Control*. The quality- diversity literature the clebran H1 (lineages diverge) is in conversation with. *(reviewer-supplied — verify)*

20. **Lehman, J., & Stanley, K.O.** (2011). *Abandoning Objectives: Evolution Through the Search for Novelty Alone*. Evolutionary Computation 19(2): 189–223. The novelty-search antecedent of the quality-diversity line. *(reviewer-supplied — verify)*

21. **McMahan, B., Moore, E., Ramage, D., Hampson, S., & y Arcas, B.A.** (2017). *Communication-Efficient Learning of Deep Networks from Decentralized Data*. AISTATS 2017. arXiv:1602.05629. The federated-learning analogue of multi-learner coordination. *(reviewer-supplied — verify)*

22. **Wang, R., Lehman, J., Clune, J., & Stanley, K.O.** (2019). *POET: Open-Ended Coevolution of Environments and their Optimized Solutions*. The asymmetric self-play framing for curriculum generation.

23. **Sukhbaatar, S., Denton, E., Szlam, A., & Fergus, R.** (2018). *Asymmetric self-play for automatic goal discovery in robotic manipulation*. The communication-through-tasks framing that the clebran curriculum channel extends.

24. **Dennis, M., Jaques, N., Vinitsky, E., Bayen, A., Russell, S., Critch, A., & Levine, S.** (2020). *Emergent Complexity and Zero-Shot Transfer via Unsupervised Environment Design*. NeurIPS 2020. PAIRED — the regret-based environment design literature.

25. **Parker-Holder, J., Jiang, N., Dennis, M., Samvelyan, M., Foerster, J., Grefenstette, E., & Rocktäschel, T.** (2022). *Evolving Curricula with Regret-Based Environment Design*. arXiv:2203.01302. ACCEL — the closest existing analogue to the clebran adversarial/co-evolved curriculum generator.

26. **Portelas, R., Colas, C., Weng, L., Hofmann, K., & Oudeyer, P.-Y.** (2020). *Automatic Curriculum Learning for Deep RL: A Short Survey*. The automatic-curriculum-learning landscape.

27. **Narvekar, S., Peng, B., Leonetti, M., Sinapov, J., Taylor, M.E., & Stone, P.** (2020). *Curriculum Learning for RL: A Survey*. JMLR.

28. **Zhu, X.** (2015). *Machine Teaching: An Inverse Problem to Machine Learning and an Approach Toward Optimal Education*. The machine-teaching literature's founding reference.

29. **Liu, W., Dai, B., Humayun, A., Tay, C., Yu, C., Smith, L., Rehg, J.M., & Song, L.** (2017). *Iterative Machine Teaching*. ICML 2017. arXiv:1705.10470.

30. **Garcia-Piqueras, M., & Hernández-Orallo, J.** (2021). *Optimal Teaching Curricula with Compositional Simplicity Priors*. ECML-PKDD 2021. The compositional-simplicity prior is the same MDL pressure that clebran's fitness function uses.

31. **Kirkpatrick, J., Pascanu, R., Rabinowitz, N., Veness, J., Desjardins, G., Rusu, A.A., et al.** (2017). *Overcoming catastrophic forgetting in neural networks*. PNAS. The

continual-learning / plasticity-loss literature the clebran 8b negative-transfer test operationalises.

32. **Schwarz, J., Czarnecki, W., Luketina, J., Grabska- Barwinska, A., Teh, Y.W., Pascanu, R., & Hadsell, R.** (2021). *Progress & Compress: A scalable framework for continual learning*. The continual-learning compress-then- expand framing.

33. **Wang, T., Zhu, J.-Y., Torralba, A., & Efros, A.A.** (2018). *Dataset Distillation*. The ML analogue of "export a tiny curriculum that teaches." arXiv:1811.10959.

34. **Sun, Y., Wang, X., Liu, Z., Miller, J., Efros, A., & Hardt, M.** (2020). *Test-Time Training with Self- Supervision*. ICLR 2020. The test-time-training antecedent of the TTT-for-ARC line. *(reviewer-supplied — verify)*

35. **Akyürek, E., Damani, M., Zweiger, A., Qiu, L., Guo, H., Pari, J., Kim, Y., & Andreas, J.** (2024). *The Surprising Effectiveness of Test-Time Training for Few-Shot Learning*. arXiv:2411.07279. The 2024 ARC Prize second-place paper award; the direct neural-ARC competitor for the clebran "small-system acquisition cost" thesis. *(reviewer-supplied — verify)*

36. **Liao, I., & Gu, A.** (2025). *ARC-AGI Without Pretraining (CompressARC)*. arXiv:2512.06104. 3rd place, ARC Prize 2025; 76K parameters, no pretraining, no ARC training-set, solves 20% of the public evaluation set by minimising a variational description length. The direct competitor on the "intelligence as compression" axis. *(reviewer-supplied — verify)*

37. **Tenenbaum, J.B.** (1999). *A Bayesian Framework for Concept Learning*. PhD thesis, MIT. The Bayesian-program- learning antecedent DreamCoder and NEO inherit. *(reviewer-supplied — verify)*

38. **Goodman, N.D., Tenenbaum, J.B., Feldman, J., & Griffiths, T.L.** (2008). *A Rational Analysis of Generic Language Based on Bayesian Learning*. The Bayesian-LOT line. *(reviewer-supplied — verify)*

39. **Schmidhuber, J.** (1997). *Discovering Neural Nets with Low Kolmogorov Complexity and its Application to Time Series Prediction*. Neural Computation 10(3): 773–795. The compression-as-cognition / MDL antecedent. *(reviewer- supplied — verify)*

40. **Ho, M., Si, C., Feng, Z., Yu, F., Yang, Y., Liu, Z., Hu, Z., & Qin, L.** (2025). *ArcMemo: Abstract Reasoning Composition with Lifelong LLM Memory*. arXiv:2509.04439. The ArcMemo paper; clebran's central-library arm (B5) is the conceptual cousin. *(First-draft citation was a hallucination; replaced with reviewer-supplied correction — verify against the arXiv listing.)*

41. **Caruana, R.** (1997). *Multitask Learning*. Machine Learning 28(1): 41–75. The negative-transfer antecedent the clebran 8b / "irrelevant teaching hurts" result extends. *(reviewer-supplied — verify)*

42. **Standley, T., Zamir, A.R., Chen, D., Guibas, L., Malik, J., & Savarese, S.** (2020). *Which Tasks Should Be Learned Together in Multi-task Learning?* ICML 2020. The task-affinity negative-transfer result. *(reviewer-supplied — verify)*

43. **Lake, B.M., & Baroni, M.** (2018). *Generalization without Systematicity: On the Compositional Skills of Sequence-to- Sequence Recurrent Networks*. ICML 2018. The SCAN compositional-generalisation benchmark. *(reviewer-supplied — verify)*

44. **Kim, N., & Linzen, T.** (2020). *COGS: A Compositional Generalization Challenge Based on Semantic Interpretation*. EMNLP 2020. arXiv:2010.05465. The COGS benchmark. *(reviewer-supplied — verify)*

45. **Ruis, L., Andreas, J., Baroni, M., Bouchacourt, D., & Lake, B.M.** (2020). *A Benchmark for Systematic Generalization in Grounded Language Understanding*. NeurIPS 2020. arXiv:2003.09760. The gSCAN benchmark. *(reviewer-supplied — verify)*

46. **ARC Prize.** (2024–2026). https://arcprize.org/. The Kaggle competition context; the clebran design's compute target is the Kaggle envelope (~12h wall-clock, 4×L4 GPUs, offline only).